Reinforcement Learning

What is RL?

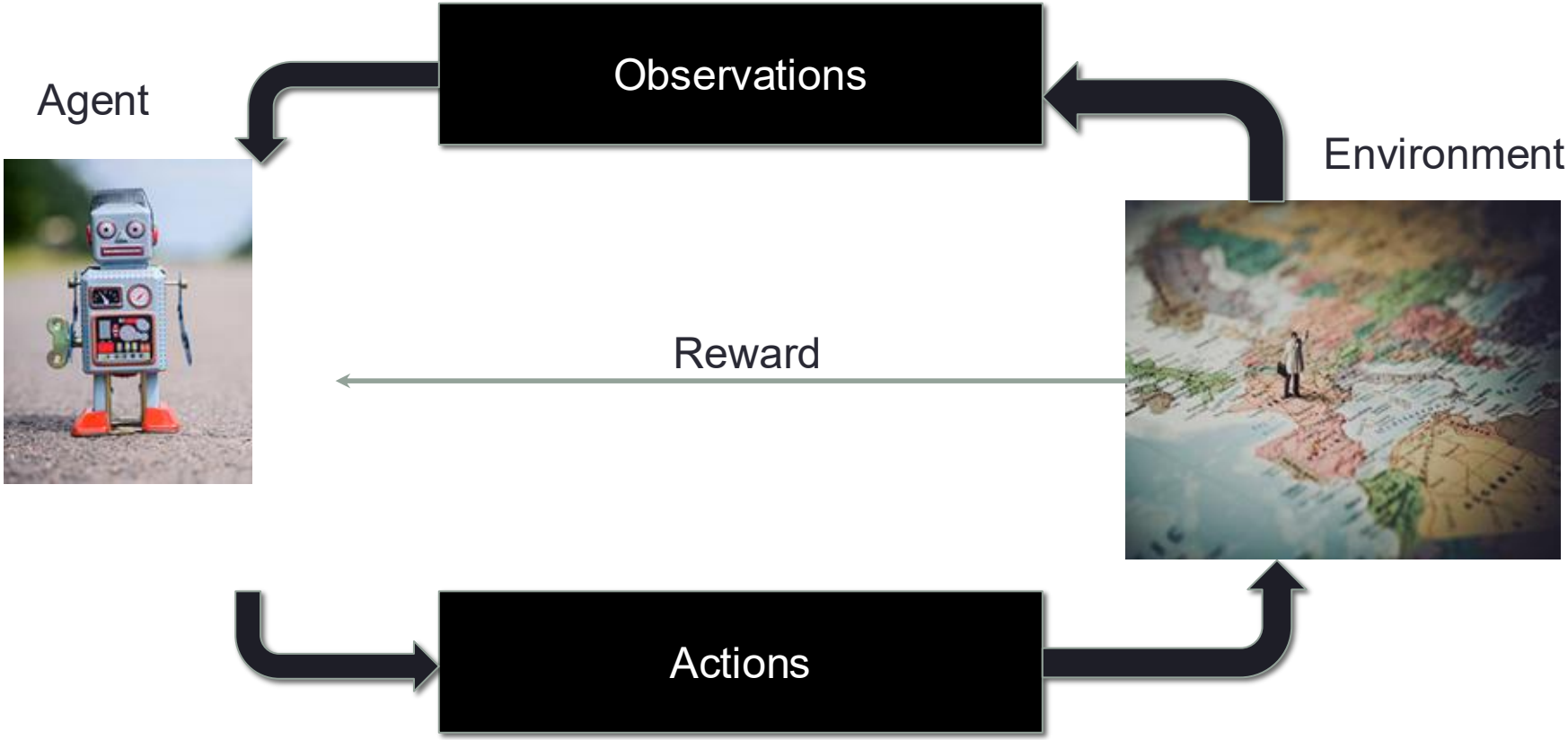
1) A problem
2) A community working on 1)
3) Methods produced by 2) which can be applicable to other problems

Benjamin Van Roy

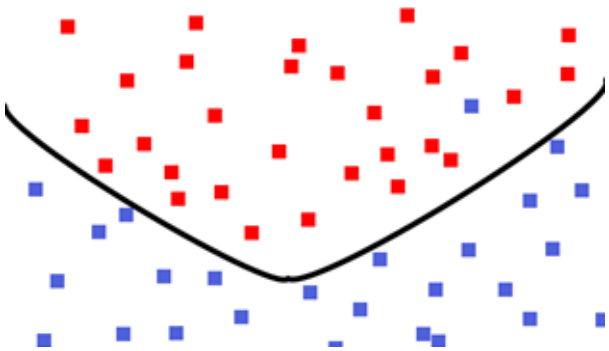
Professor at Stanford University; Research Lead at DeepMind, Mountain View

Topic: Reinforcement Learning

RL problem

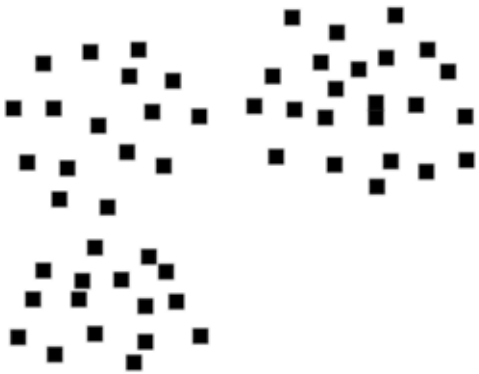


3 Modes of Learning

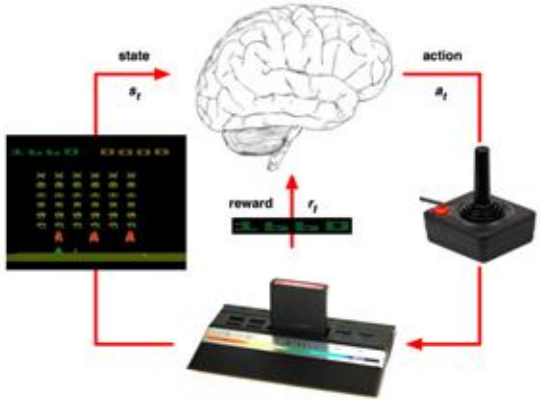


Supervised Learning

Reinforcement Learning

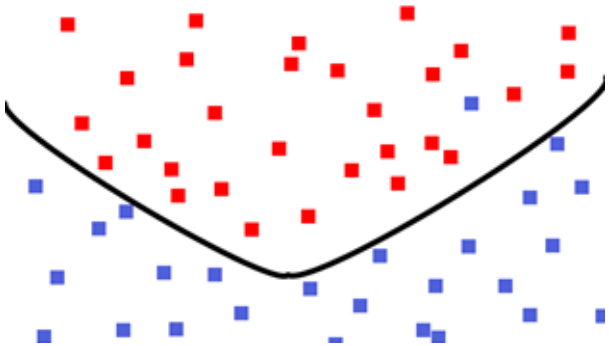


Unsupervised Learning



3 Modes of Learning

Supervised Learning

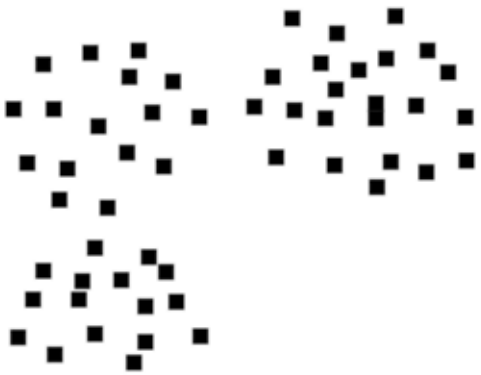


- Observe:
  - $(x_1, y_1), (x_2, y_2), \ldots$
- Objective:
  - Input an unseen x_{new}
  - What is y_{new}?

3 Modes of Learning

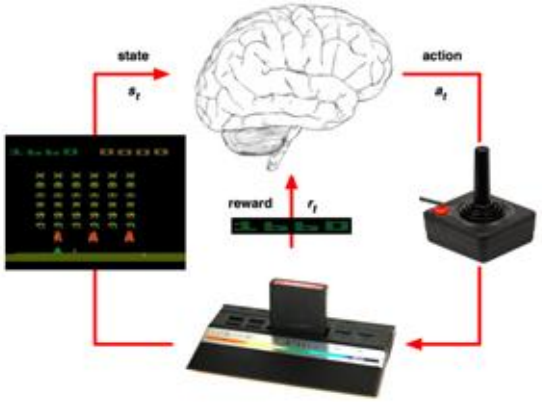
Unsupervised Learning

- Observe:
  - $x_1, x_2, x_3, x_4, \ldots$
- Objective:
  - What is P(x) ?
  - What is a *good* representation of x ?
  - What can we learn from P(x) ?



3 Modes of Learning

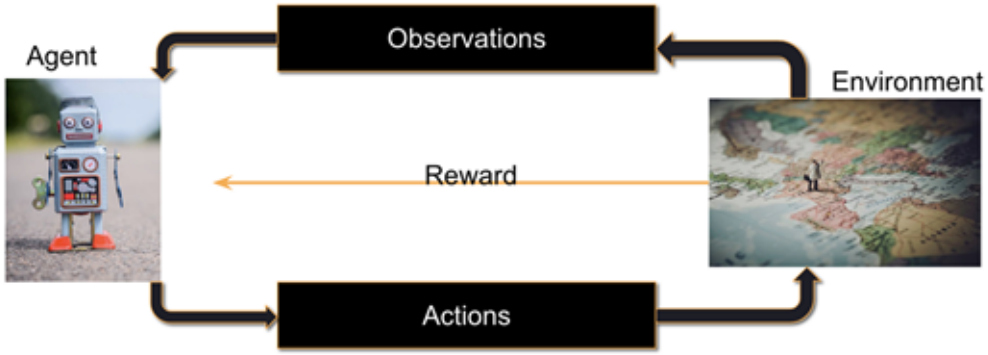
Reinforcement Learning (RL)



- Observe:
  - The states (x_1, x_2, x_3, …)
  - The reward (r_1, r_2, r_3, …)
- Can also take actions
  - a_1 , a_2 , a_3 , …
- What are the best actions?
  - Such that we will receive highest accumulative rewards

Difference between RL and other modes of learning

- Sequential decisions
- You have a goal vs
  You have means to get there
- No concept of “training set” and
  “test set”
- “Passive” vs “Active” learning



RL and Artificial General Intelligence

Yann LeCun

March 14, 2016 ·

Follow

Statement from a Slashdot post about the AlphaGo victory: "We know now that we don't need any big new breakthroughs to get to true AI"

That is completely, utterly, ridiculously wrong.

As I've said in previous statements: most of human and animal learning is unsupervised learning. If intelligence was a cake, unsupervised learning would be the cake, supervised learning would be the icing on the cake, and reinforcement learning would be the cherry on the cake. We know how to make the icing and the cherry, but we don't know how to make the cake.

We need to solve the unsupervised learning problem before we can even think of getting to true AI. And that's just an obstacle we know about. What about all the ones we don't know about?

#deeplearning #AI #AlphaGo

This is before he coined the term self-supervised learning for supervised learning

The Yann Lecunn's cake

How Much Information is the Machine Given during Learning?

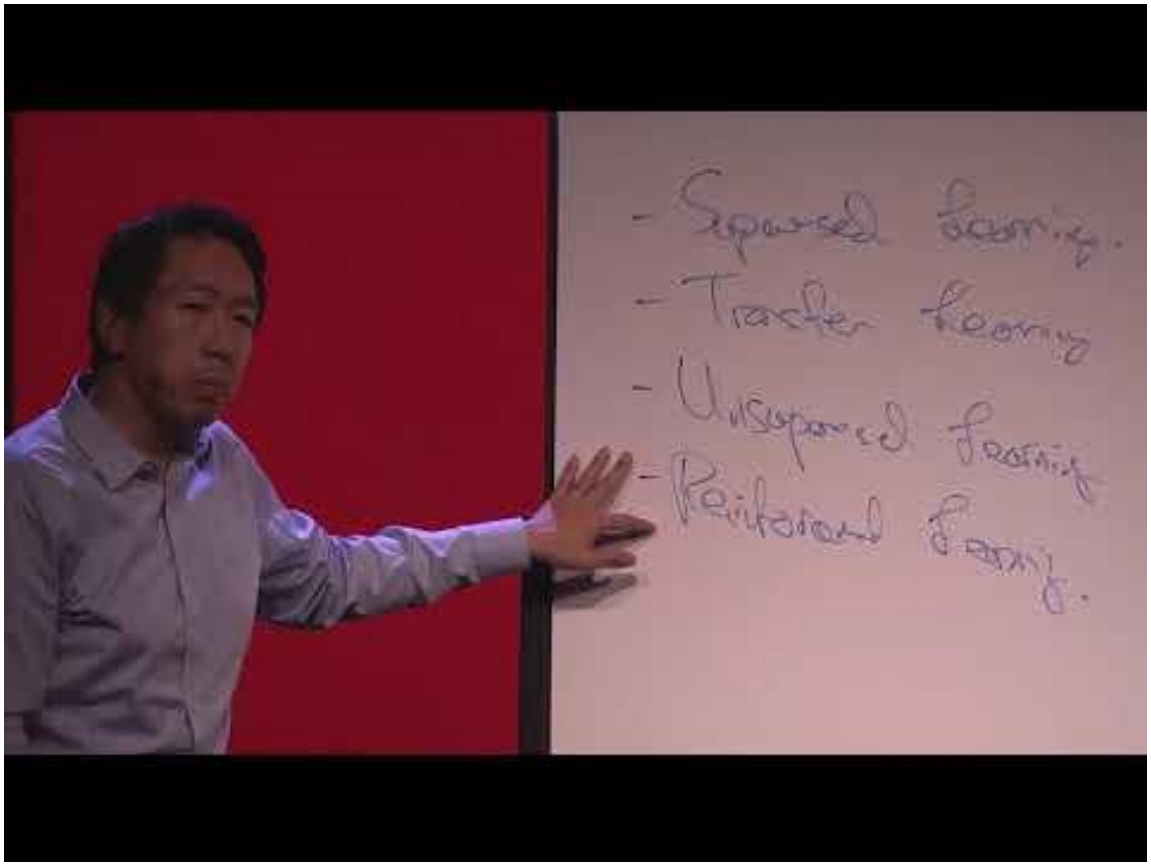
Y. LeCun

- **"Pure" Reinforcement Learning (cherry)**
  - The machine predicts a scalar reward given once in a while.
  - **A few bits for some samples**
- **Supervised Learning (icing)**
  - The machine predicts a category or a few numbers for each input
  - Predicting human-supplied data
  - **10→10,000 bits per sample**
- **Self-Supervised Learning (cake génoise)**
  - The machine predicts any part of its input for any observed part.
  - Predicts future frames in videos
  - **Millions of bits per sample**

https://drive.google.com/file/d/17w443t_5Atnwnu-iOrHKUPFik1pThyhx/view

RL and $$$

"The excitement and PR hype behind reinforcement learning is a bit disproportionate relative to the economic value it's creating today" - Andrew Ng



https://www.youtube.com/watch?v=NQK4ZY_gwKI&feature=youtu.be&t=1m16s

RL use cases

Go, chess, starcraft, dota, poker

Finance (https://www.jpmorgan.com/global/LOXM)...

Robotics...but...

https://research.googleblog.com/2016/03/deep-learning-for-robots-learning-from.html

https://towardsdatascience.com/applications-of-reinforcement-learning-in-real-world-1a94955bcd12
https://www.oreilly.com/ideas/practical-applications-of-reinforcement-learning-in-industry

RL use cases

Data center and resource management (https://people.csail.mit.edu/alizadeh/papers/deeprm-hotnets16.pdf System configuration http://ranger.uta.edu/~jrao/papers/ICDCS09.pdf DRAM controller https://ieeexplore.ieee.org/abstract/document/4556714/ Recommender (Bandits) https://people.cs.umass.edu/~pthomas/papers/Barto2017.pdf)

Ad bidding (https://arxiv.org/abs/1701.02490)

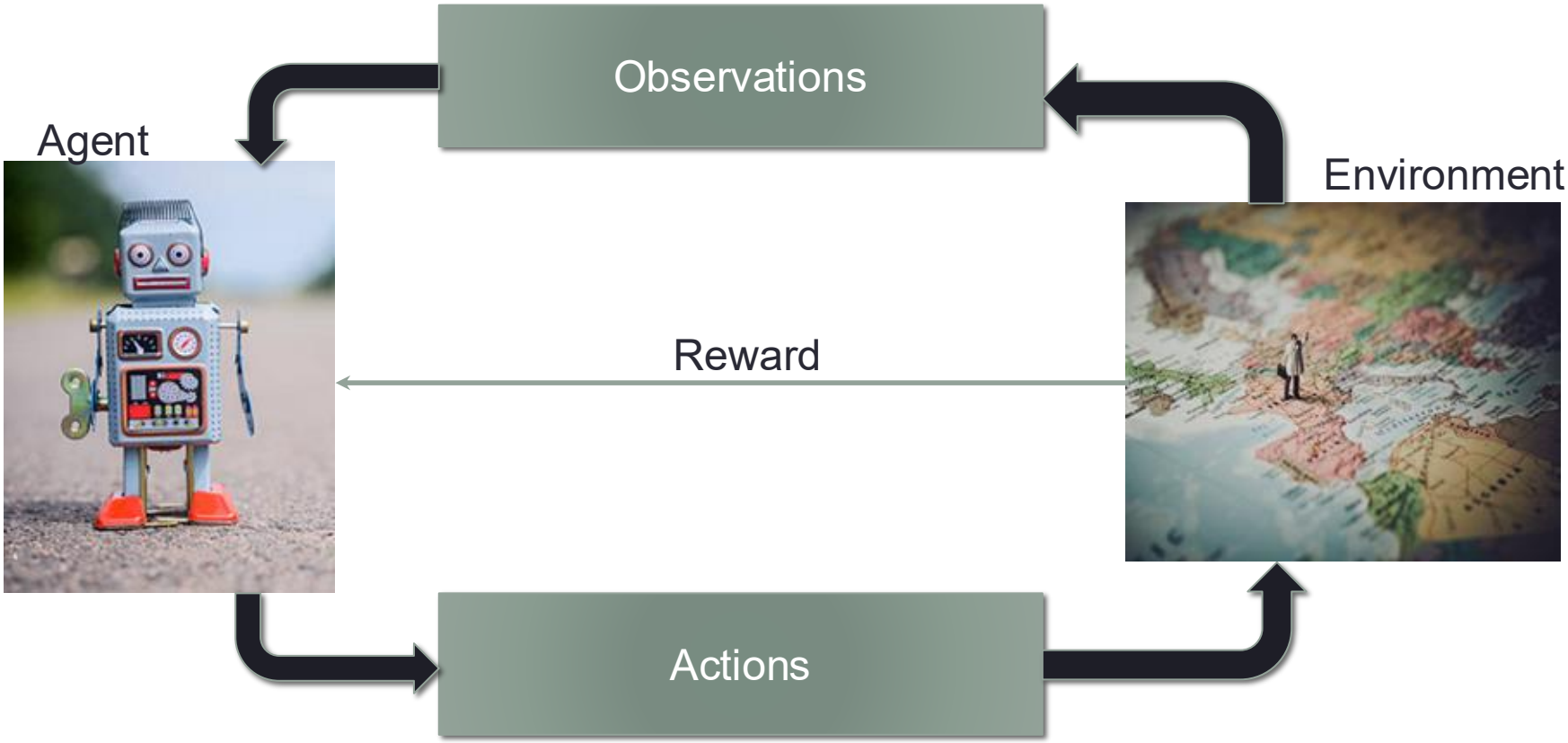
Chemistry (https://pubs.acs.org/doi/full/10.1021/acscentsci.7b00492)

ChatGPT

Some other tasks that use algorithms from RL to help perform model training (autoML, REINFORCE)

ACS Central Science

RL framework



Learning through trial and error

RL framework

Reward (r_t)

State (s_t)

Action (a_t)

$$\begin{tikzcd}
\rightarrow \text{Env.} \arrow[r, "s_t"] \arrow[d] & \text{Agent} \arrow[r, "a_t"] & \text{Env} \arrow[r, "s_{t+1}"] \arrow[d] & \text{Agent} \arrow[r, "a_{t+1}"] & \text{Env} \arrow[r, "s_{t+2}"] \arrow[d] & {} \\
r_t & & r_{t+1} & & r_{t+2} &
\end{tikzcd}$$

Markov property

$$a_t = A(s_t)$$

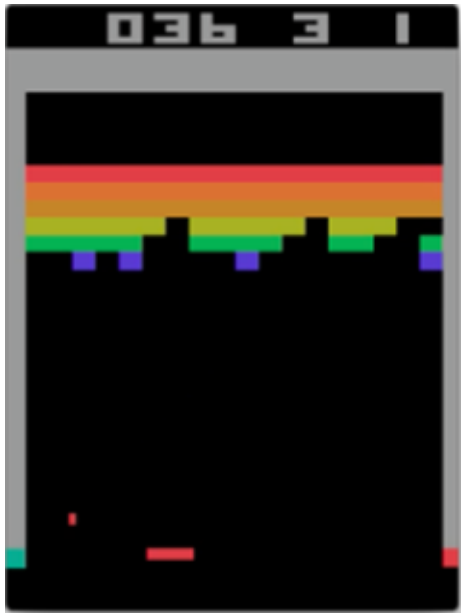
$$r_{t+1} = R(s_t, a_t)$$

$$s_{t+1} = S(s_t, a_t)$$

Rewards-based learning

- Maximise the rewards
- Can we design any desired behaviour with reward?

$R_t = \Delta\text{distance}$



$R_t = \text{score}$

$$R_T = \begin{cases} 1 & \text{, win} \\ -1 & \text{, lose} \end{cases}$$

The Environment

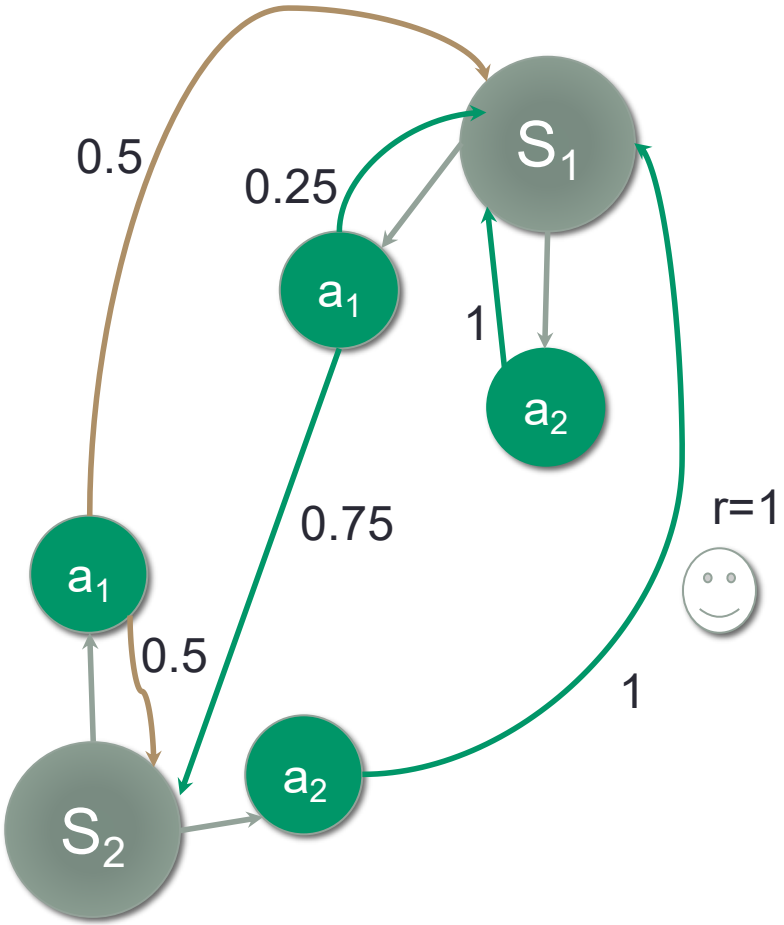
How can we model the environment?

Markov Decision Process (MDP)

- **S,A,P,R**,γ
- **S** – Set of states
- **A** – Set of actions
- **P** – Transition between states given an action

$$P^{a}_{s,s'} = \mathrm{Prob}[s_{t+1}=s' \mid s_t=s,\ a_t=a]$$

- **R** – Rewards associated with actions and states
- γ – Discount factor



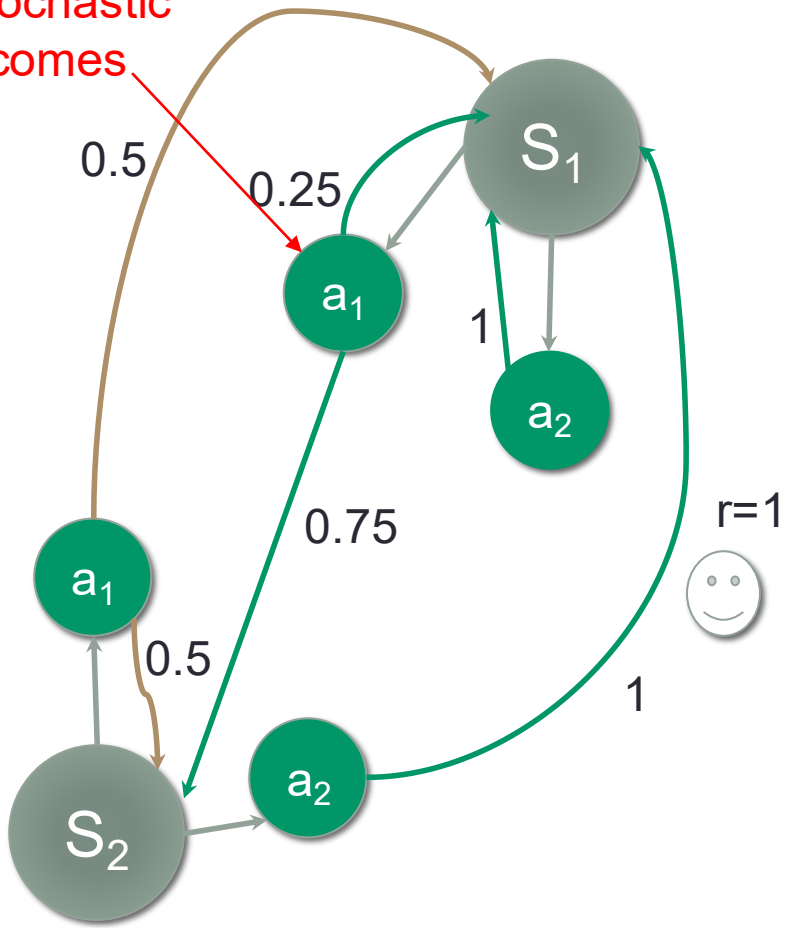
Markov Decision Process (MDP)

Environment could be stochastic
One action, multiple outcomes

- **S,A,P,R**,γ
- **S** – Set of states
- **A** – Set of actions
- **P** – Transition between states given an action

$$P^{a}_{s,s'} = \mathrm{Prob}[s_{t+1}=s' \mid s_t=s,\ a_t=a]$$

- **R** – Rewards associated with actions and states
- γ – Discount factor



Markov Property

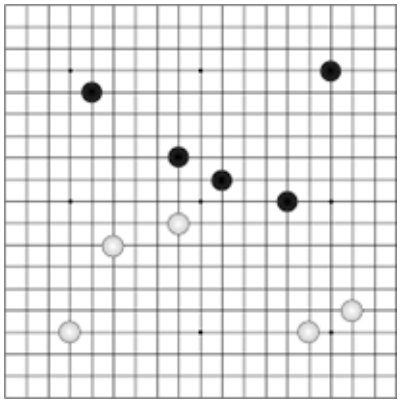
$$p(s_{t+1} | s_t, a_t)$$

- s_{t+1} depends only on s_t
- not s_{t-1}, not anything before
- this simplifies our situation!

Fog of war

But, is it true in every case?

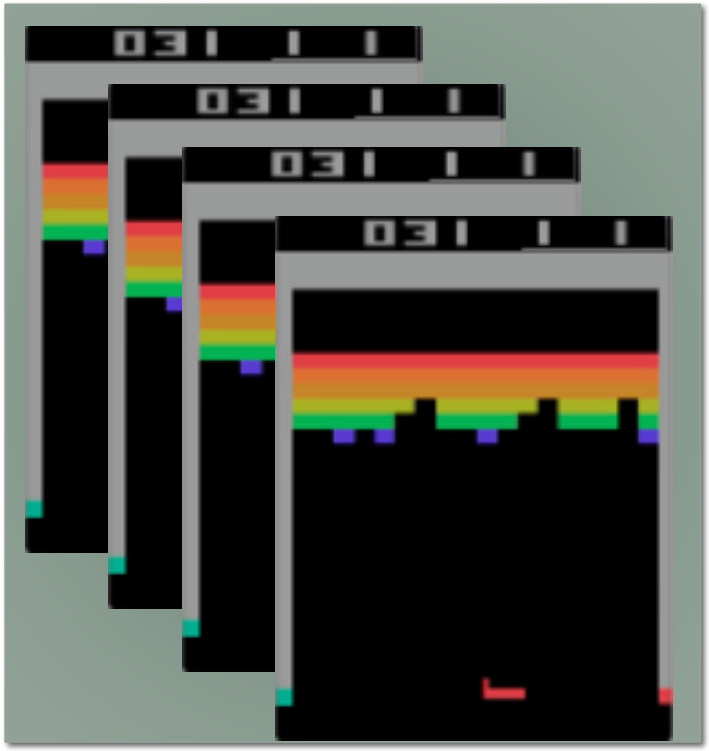
- It depends on your observed state
  - Fully observable state ✔
  - Partially observable state ✖



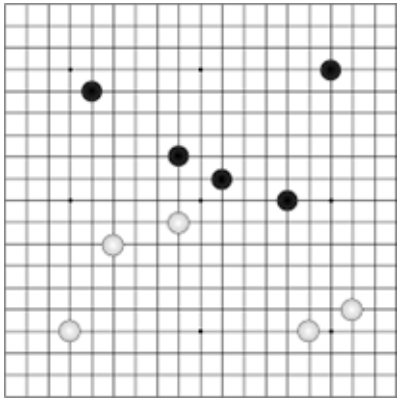
Fully observable

Fully Observable State

- Fully observable state: All information from the past is captured in the current state



For Go, a board position
For simple video games, stack multiple frames



The Agent

Policy

- Policy = a mapping from a state to an action
- Objective of RL is to find the ‘’*optimal*” policy!

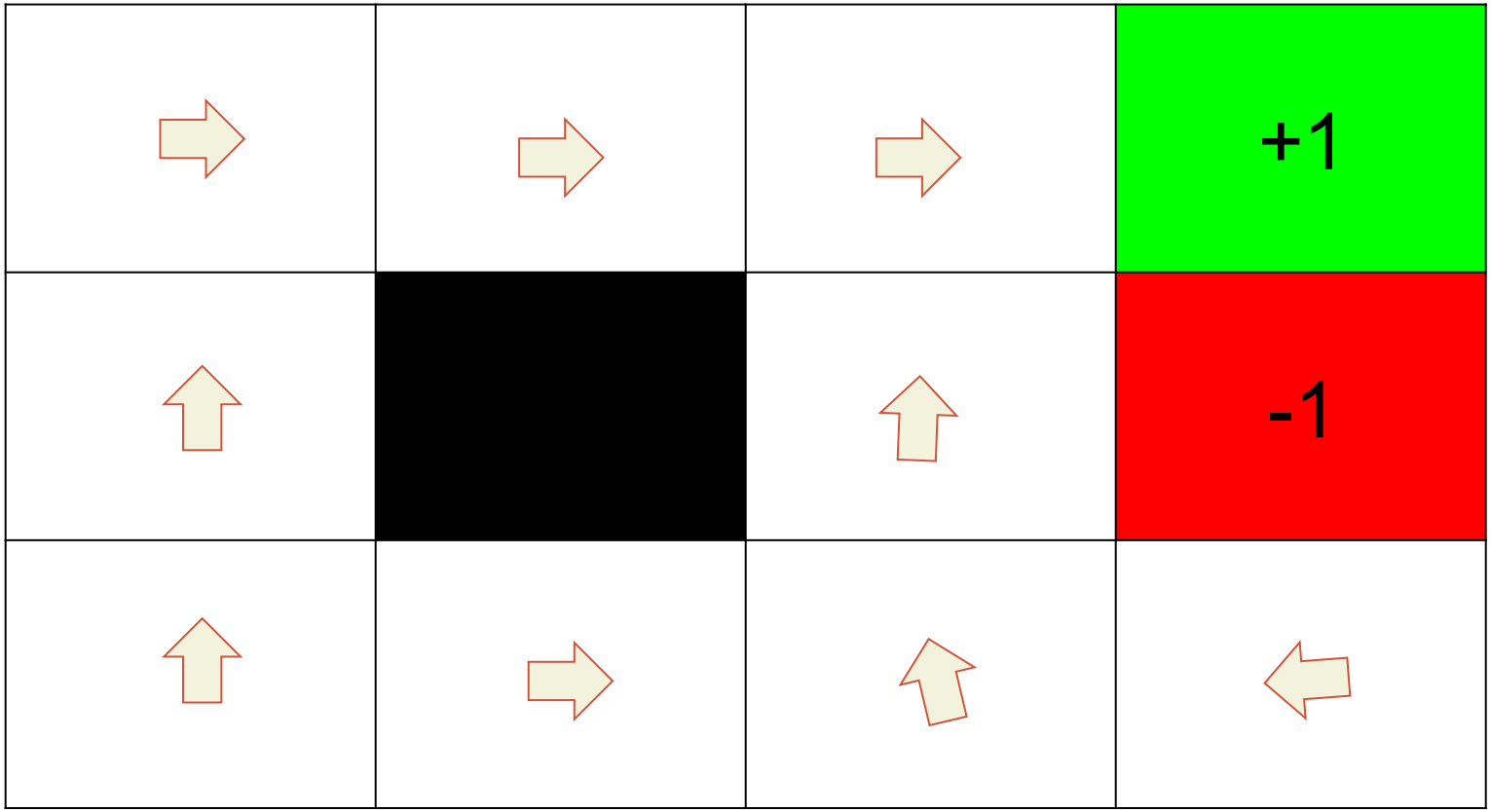
$$a = \pi(s)$$

$$a \sim \pi(s)$$

Can be either deterministic or stochastic

Policy

Example: tabular policy



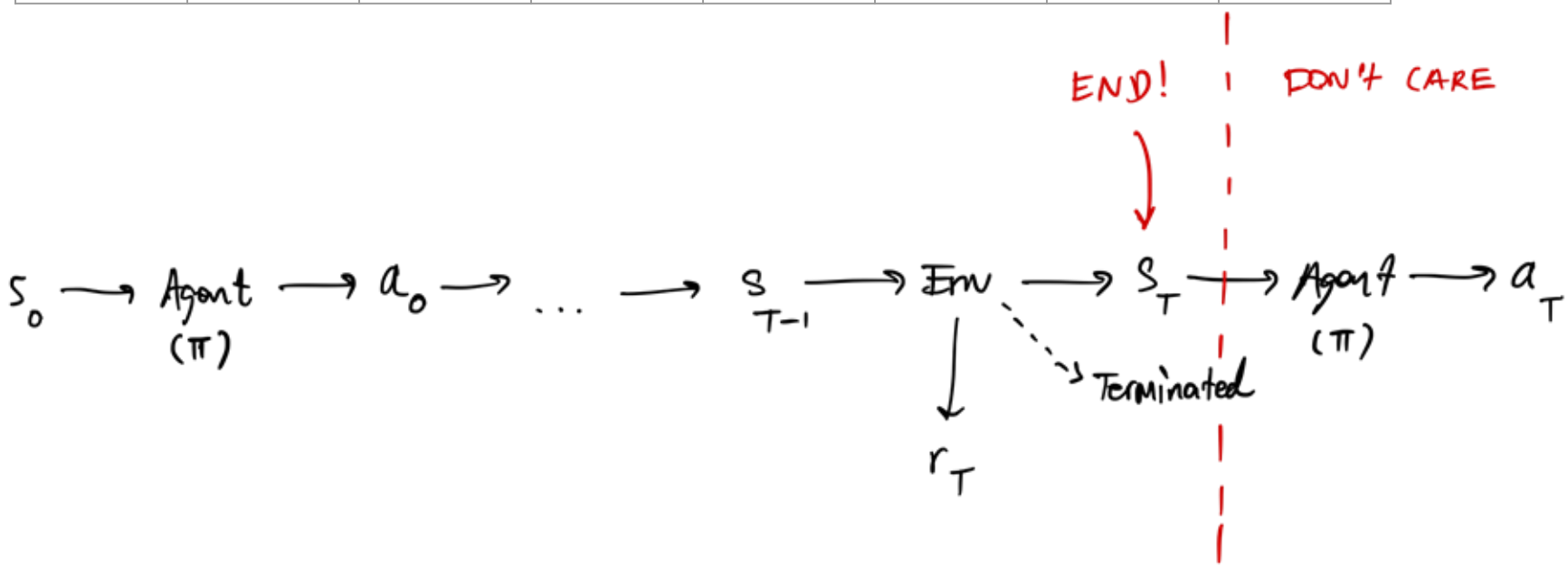
Learning

How do we find the best policy ?

Rollout (Our data)

Time	0	1	2	3	...	T-1	T
S	S_0	S_1	S_2	S_3	...	S_{T-1}	S_T
A	A_0	A_1	A_2	A_3	...	A_{T-1}	A_T
R	R_0	R_1	R_2	R_3	...	R_{T-1}	R_T
Done	0	0	0	0	...	0	1

Don't care

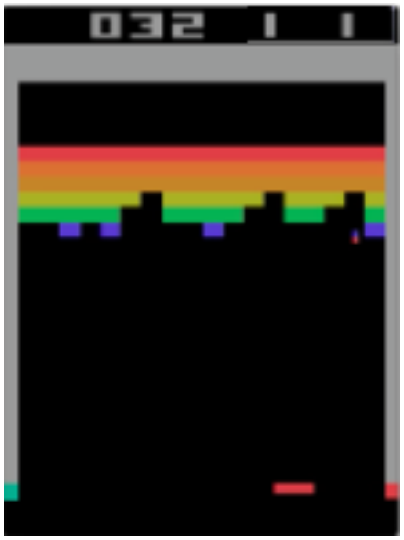


Return (Cumulative rewards)

- Return = cumulative rewards with discount

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots = \sum_{t'=t}^{\infty} \gamma^{t'-t} r_t$$

$r_t = 0$



$r_{t+10} = 1$

$r_{t+34} = -1$

What is learning?

- Use data to find/search for the best policy

What is the best policy?

- Policy that give us the highest expected return!

$$G = r_0 + \gamma r_1 + \gamma^2 r_2 + \ldots$$

$$J(\pi) = E_\pi[G]$$

Return under a policy (G^{π})

$$\pi(s_0) \quad \pi(s_1) \quad \pi(s_2) \quad \pi(s_3) \quad \pi(s_4)$$

$$s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, r_3, s_3, a_3, r_4, s_4, a_4$$

$$G_0 = r_1 + r_2 + r_3 + r_4 + \dots$$

$$G_1 = r_2 + r_3 + r_4 + \dots$$

$$G_2 = r_3 + r_4 + \dots$$

$$G_3 = r_4 + \dots$$

* THIS IS JUST ONE TRY!
ANOTHER MIGHT YIELD DIFFERENTLY

Expected Return

How good “on average” is our return?

Statistical expectation

For value G_0

$$E_\pi[G_0]$$

Following policy π

- Start
- Play with π
- $G_0 = \sum_{t=0}^{T} r_t$
- Retry many times (∞)
- Average G_0

A naive learning method

1. Initialise a policy randomly
2. Evaluate the policy by running that policy multiple times
   a. which we then collect the returns of all the runs
3. Randomly initialise another policy
4. Evaluate the new policy
5. Keep the policy that have a higher expected return
6. Repeat 3-5

Intuitive. But very inefficient!

How to make it more efficient?

Categories of RL methods

Value-based vs Policy-based

Model-based vs Model-free

On-policy vs Off-policy

Trade-off on a spectrum

Valued-based RL

Q-learning algorithm

- Let's define a state value as
  - Expectation of the return after visit s and follow π

$$V^{\pi}(s) = E_{\pi}[G_t | s_t = s]$$

- Let's define a state-action value (Q-value) as
  - Expectation of the return after visit a state s, take action a

$$Q^{\pi}(s, a) = E_{\pi}[G_t | s_t = s, a_t = a]$$

$$V^{\pi}(s) = E_{\pi(s)}[Q^{\pi}(s, \pi(s))]$$

Q-learning algorithm

- There exist an optimal value function associate with an optimal policy,

$$V^*(s) = \max_\pi V^\pi(s) \quad \forall s \in S$$

- The optimal policy is the policy that achieves the highest value for every state

Q-learning algorithm

- It follows that

$$V^*(s) = \max_a [Q^*(s,a)]$$

- and ..

$$\pi^*(s) = argmax_a Q^*(s,a)$$

- Optimal actions can be found indirectly through Q-value

Value-based learning

2 Steps

- Improve Q/V
- Improve the policy

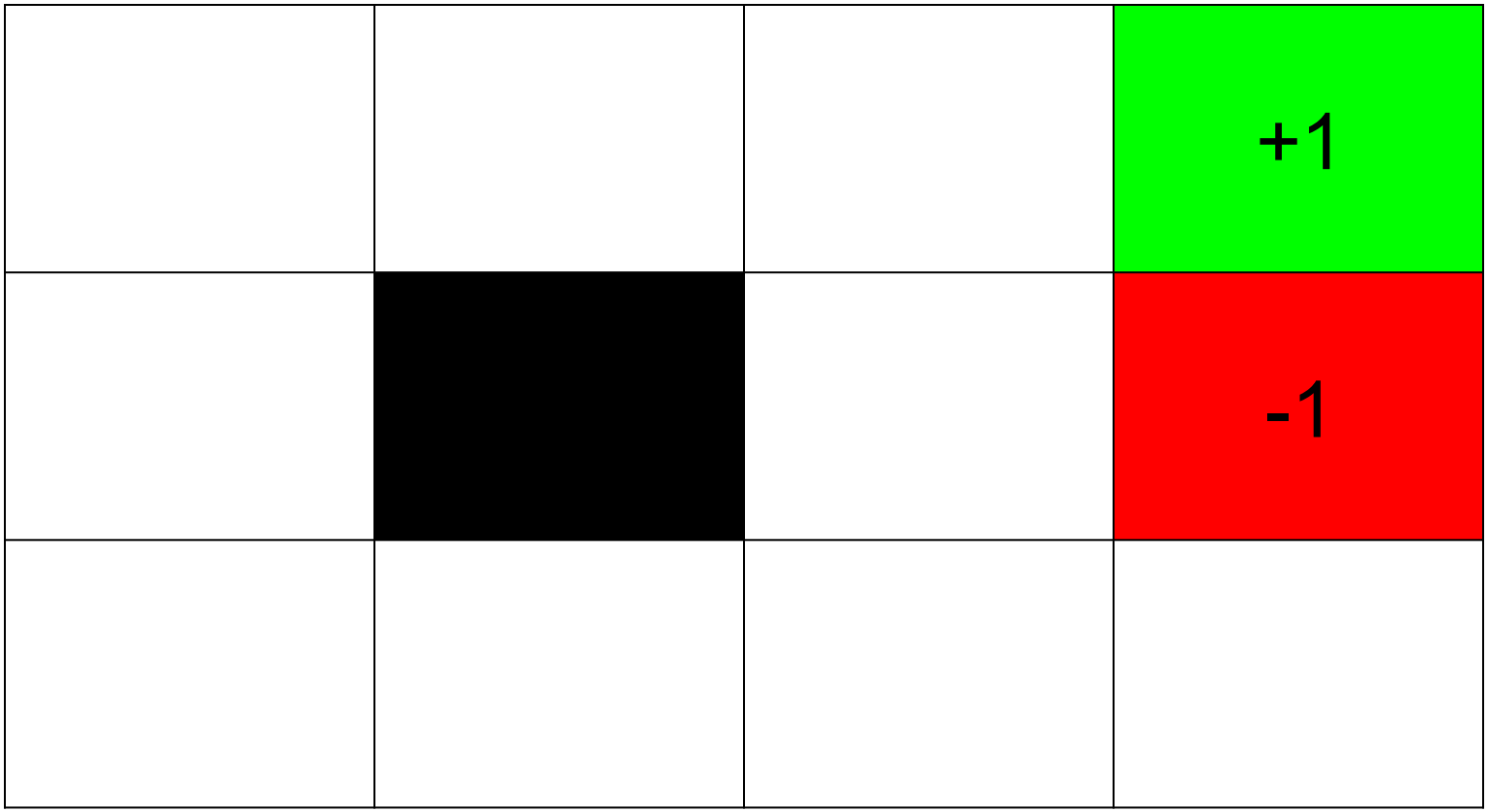
How to learn Q/V?

- Monte-Carlo
- Bootstrap

How to improve the policy?

- Follows the best Q/V

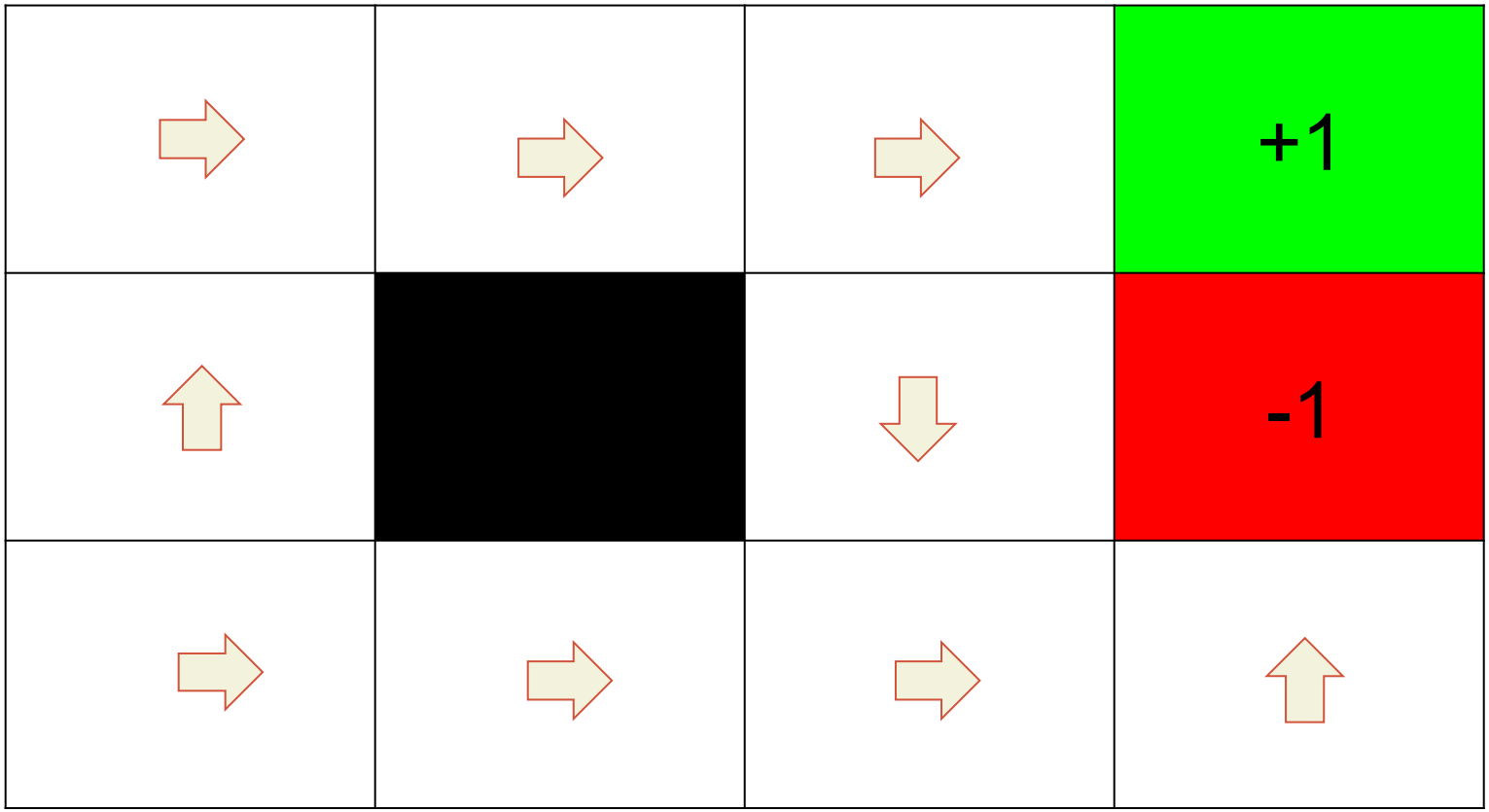
Example, Tabular Q-learning



Monte-Carlo Estimator

Monte-Carlo compute Q estimates using an average of rollouts based on a policy.

- Initialise π

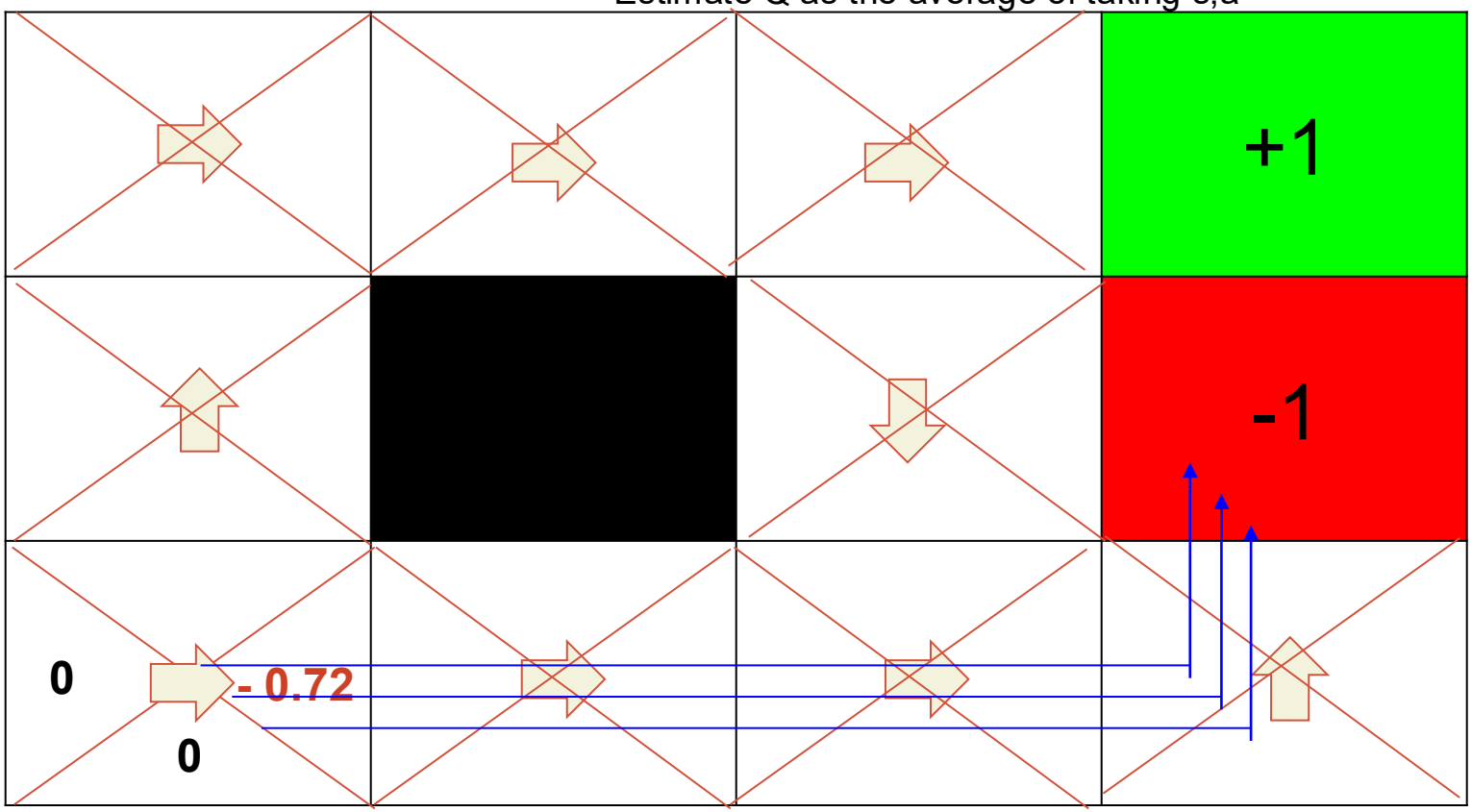


Monte-Carlo Estimator

$$Q^{\pi}(s,a) \approx r_0 + \gamma r_1 + \gamma^2 r_2 + .. + \gamma^n r_n$$

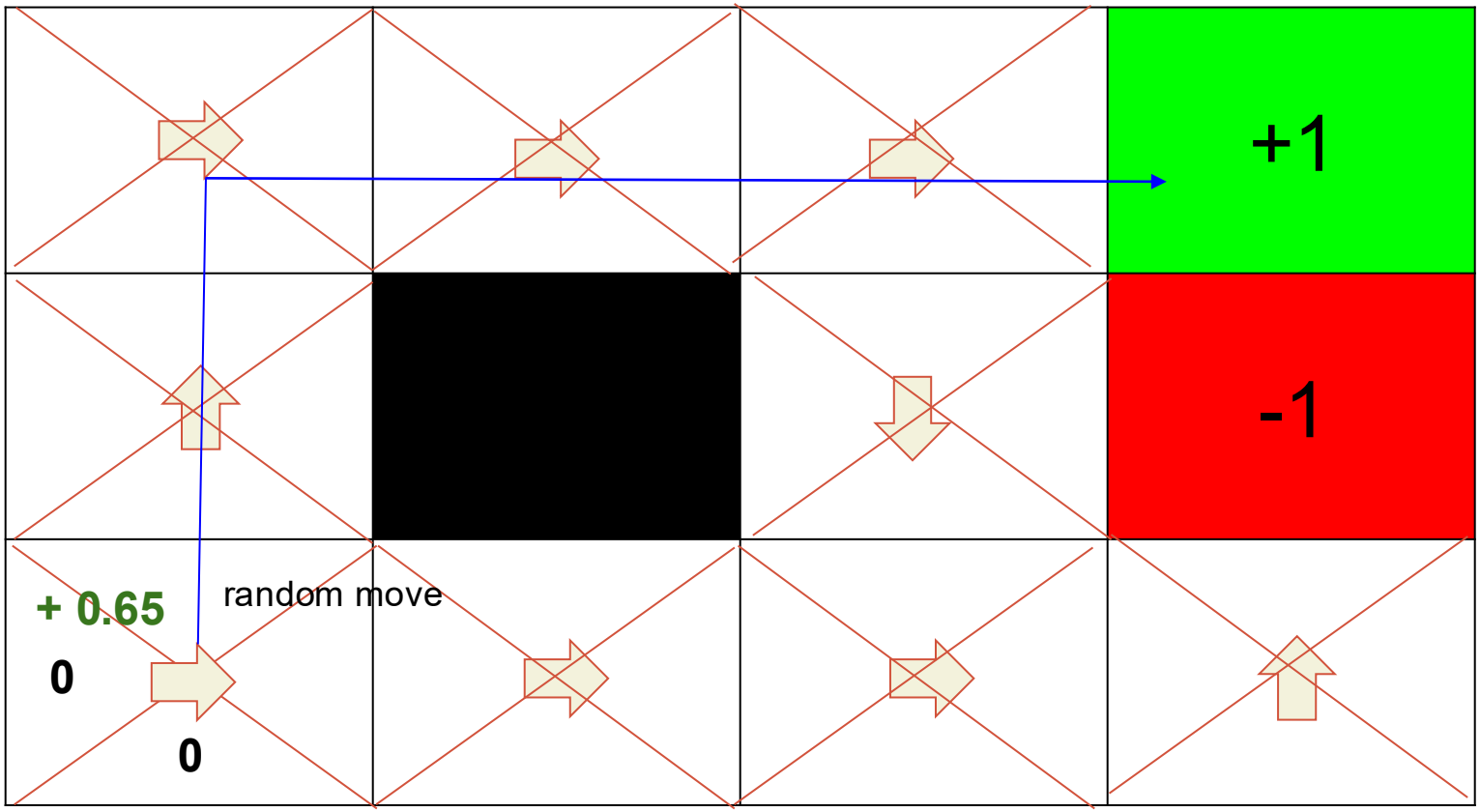
$$\gamma = 0.9$$

Running simulations until the end.
Estimate Q as the average of taking s,a



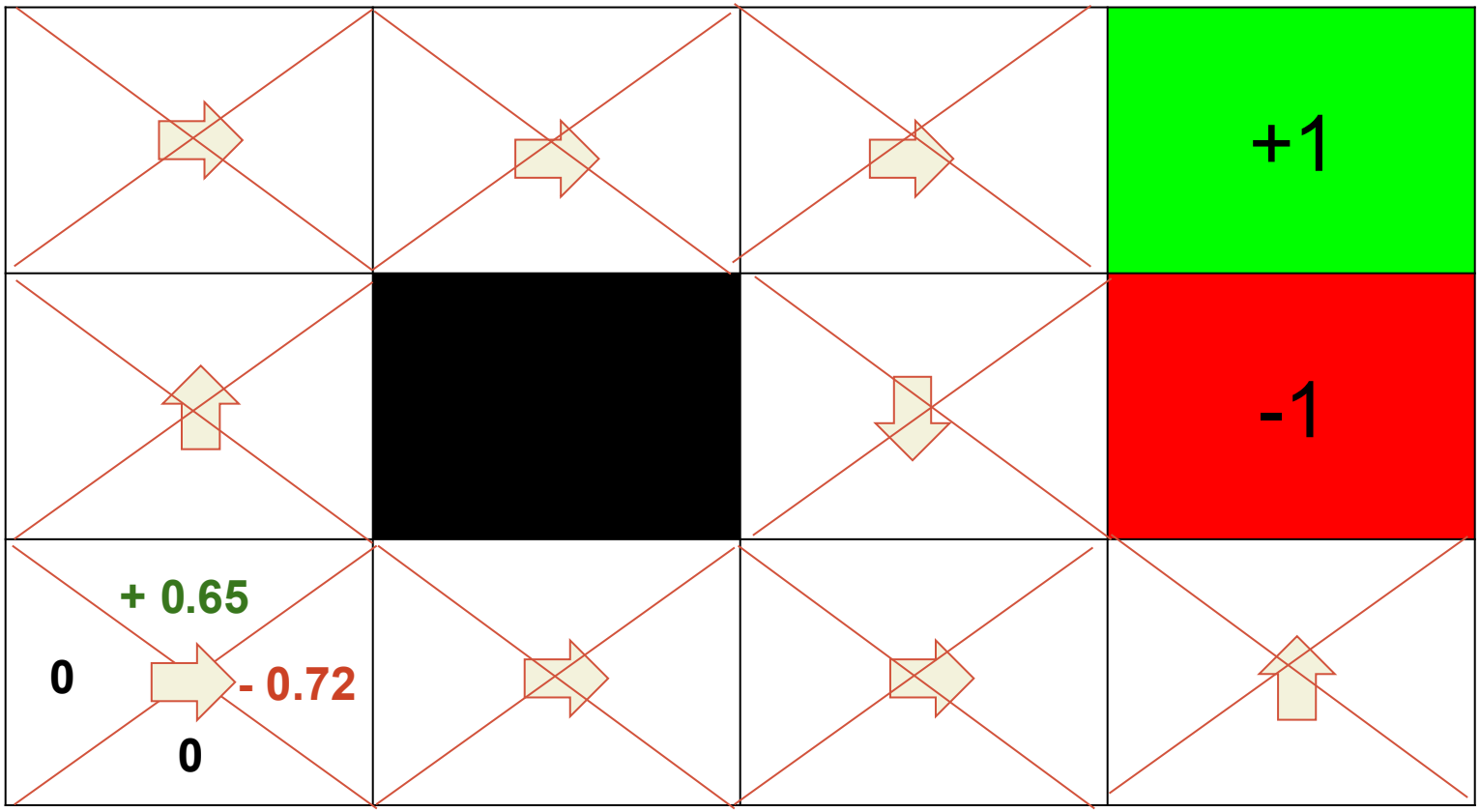
Exploration

Randomly explore sometimes to get better solutions
Have epsilon probability to take a random move. Take current policy move with 1-epsilon probability.



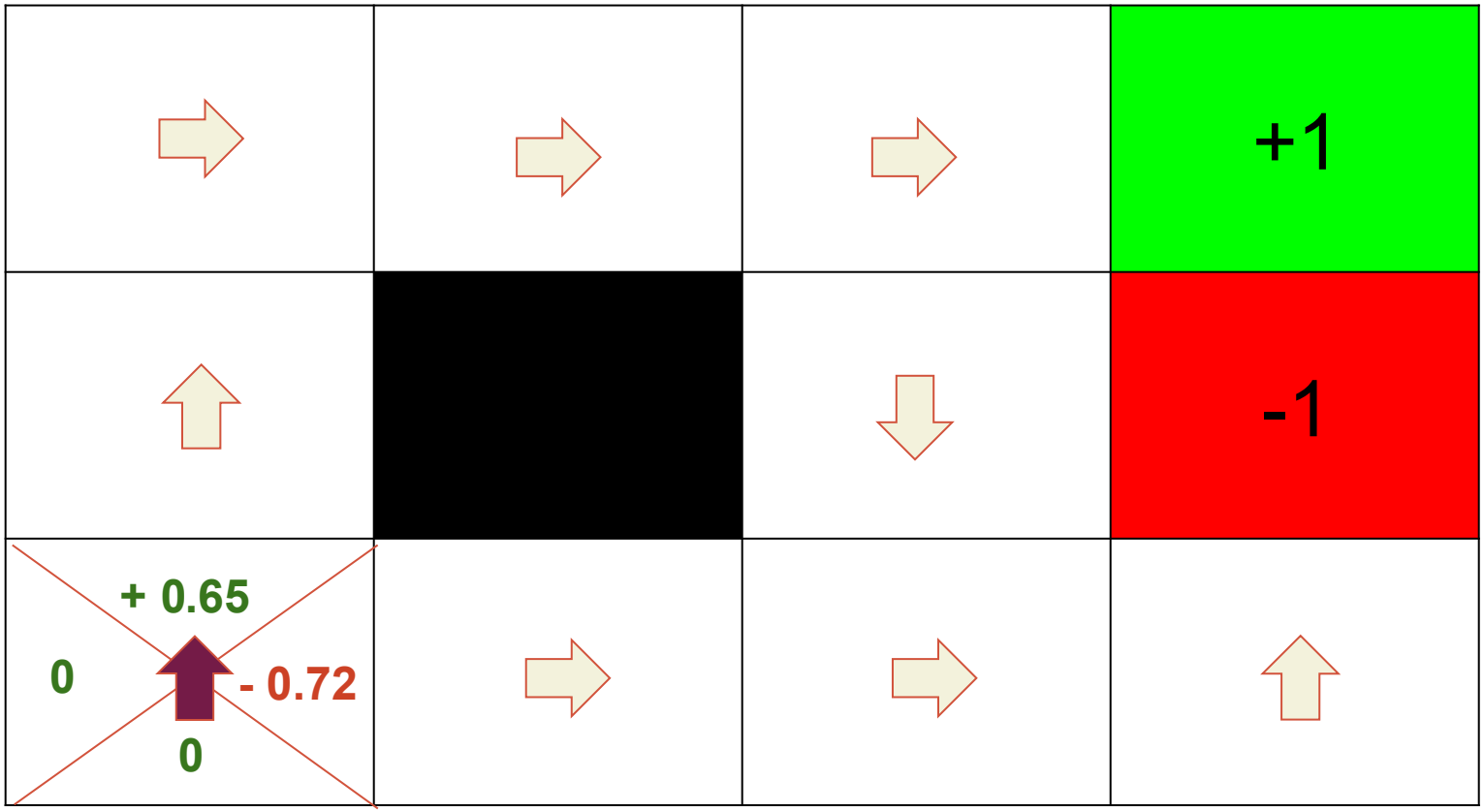
Monte-Carlo Estimator

Running many simulations until the end to estimate Q(s,a)



Policy Improvement

$$\pi'(s) = argmax_a Q(s,a)$$



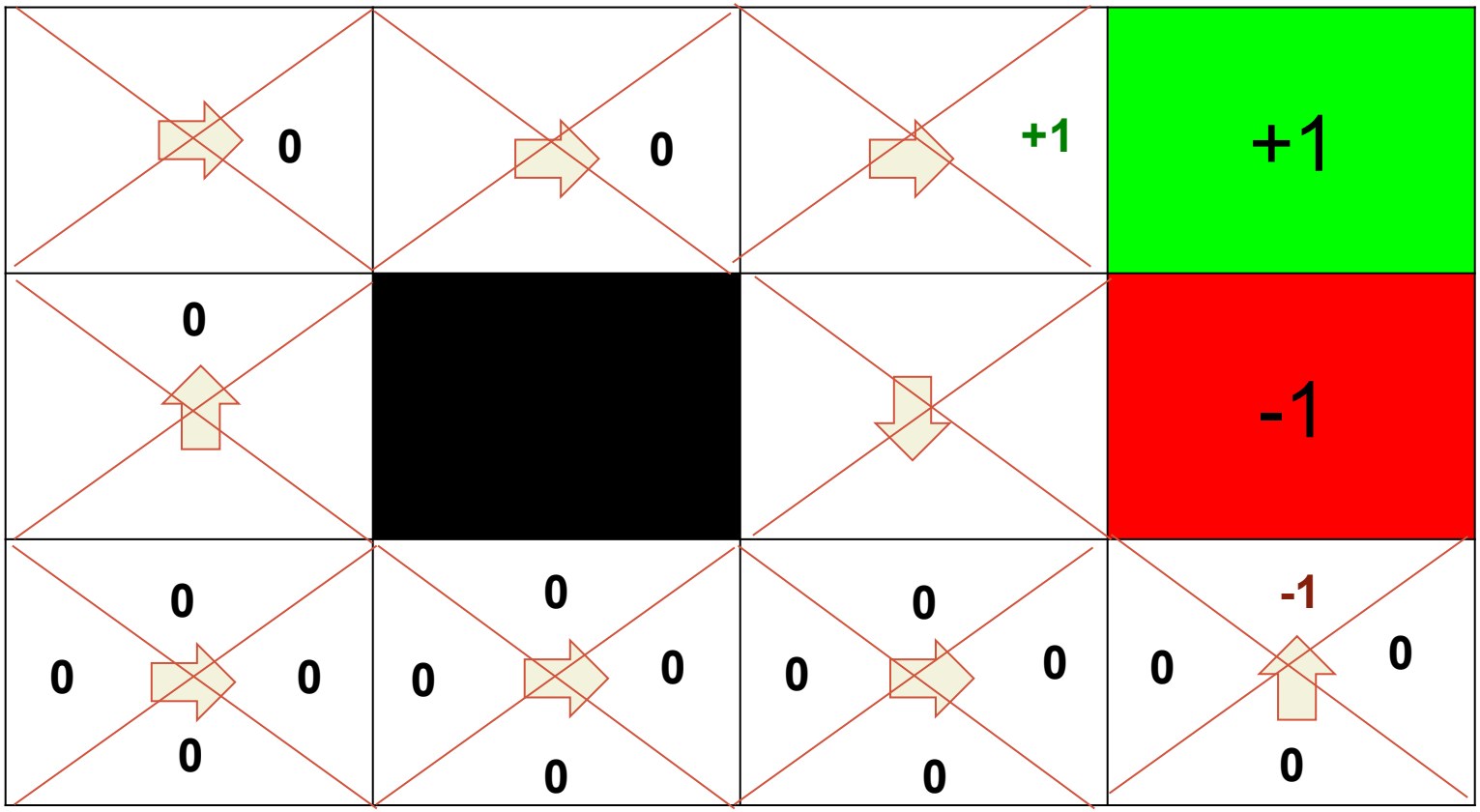
Bootstrap Estimator

Bootstrap updates Q estimates using its own predictions rather than using complete rollouts.

current reward

estimate Q of subsequent states

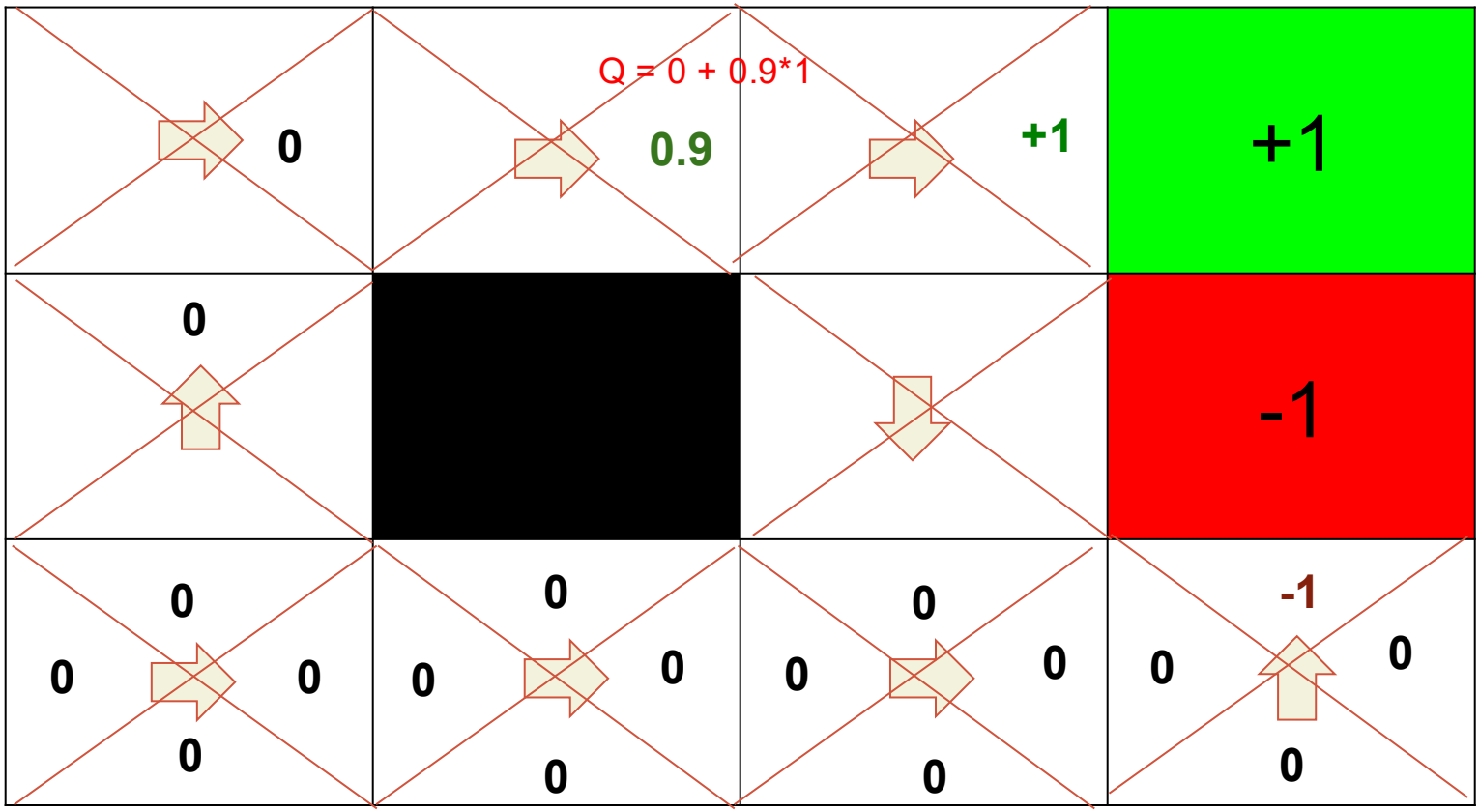
$$Q(s,a) \approx r_0 + \gamma \max_b Q(s',b)$$



Bootstrap Estimator

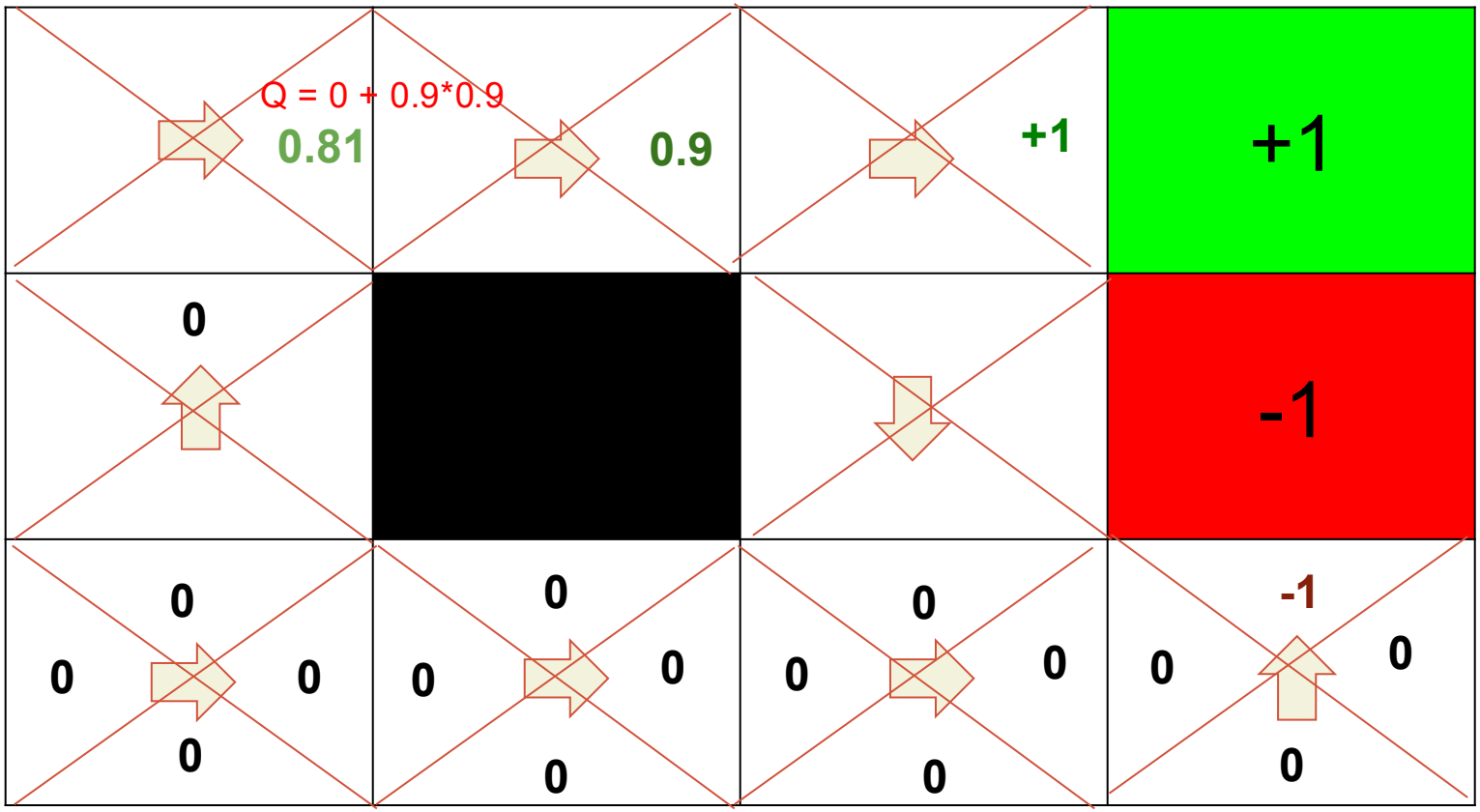
bellman equation

$$Q(s,a) \approx r_0 + \gamma \max_b Q(s',b)$$



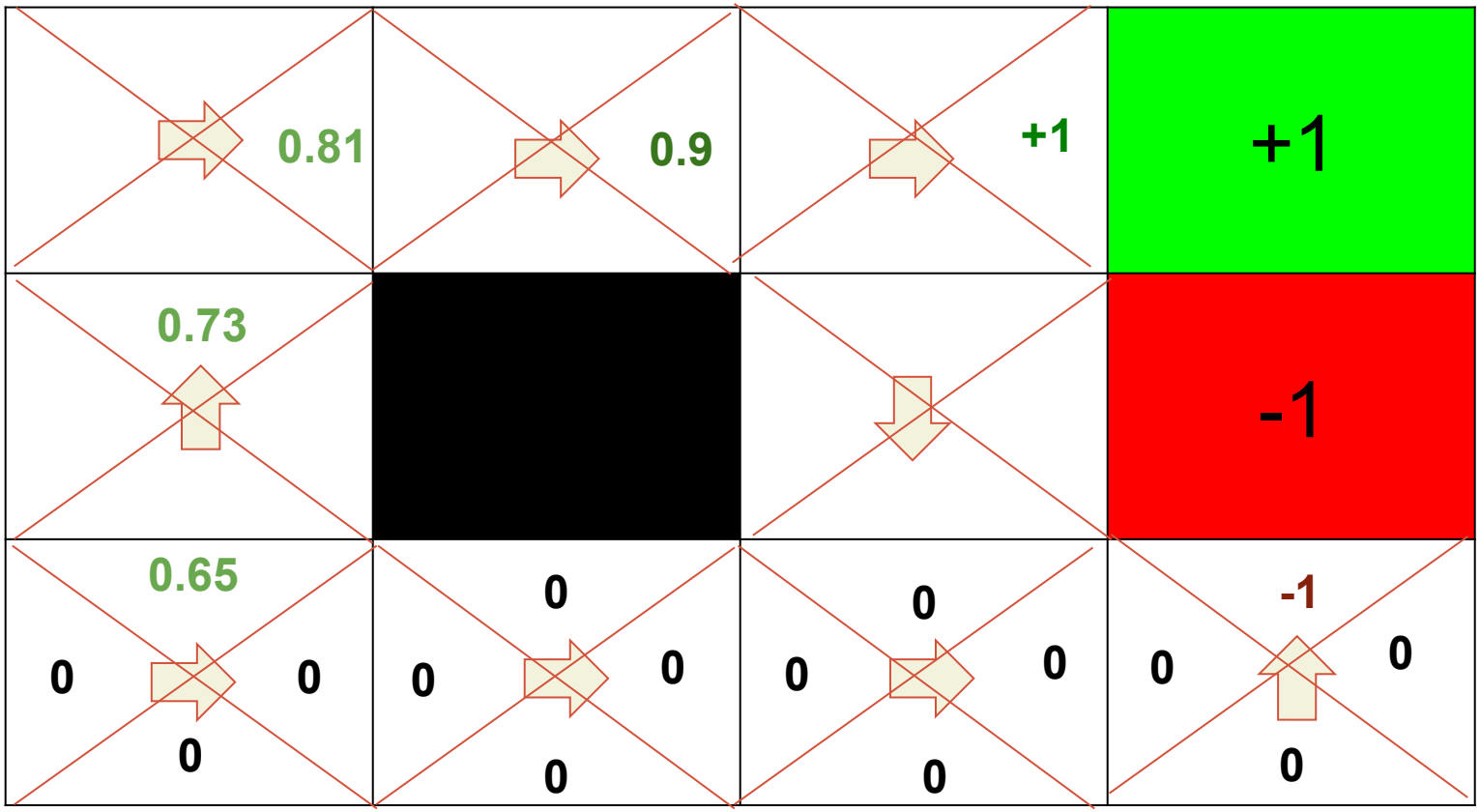
Bootstrap Estimator

$$Q(s,a) \approx r_0 + \gamma \max_b Q(s',b)$$

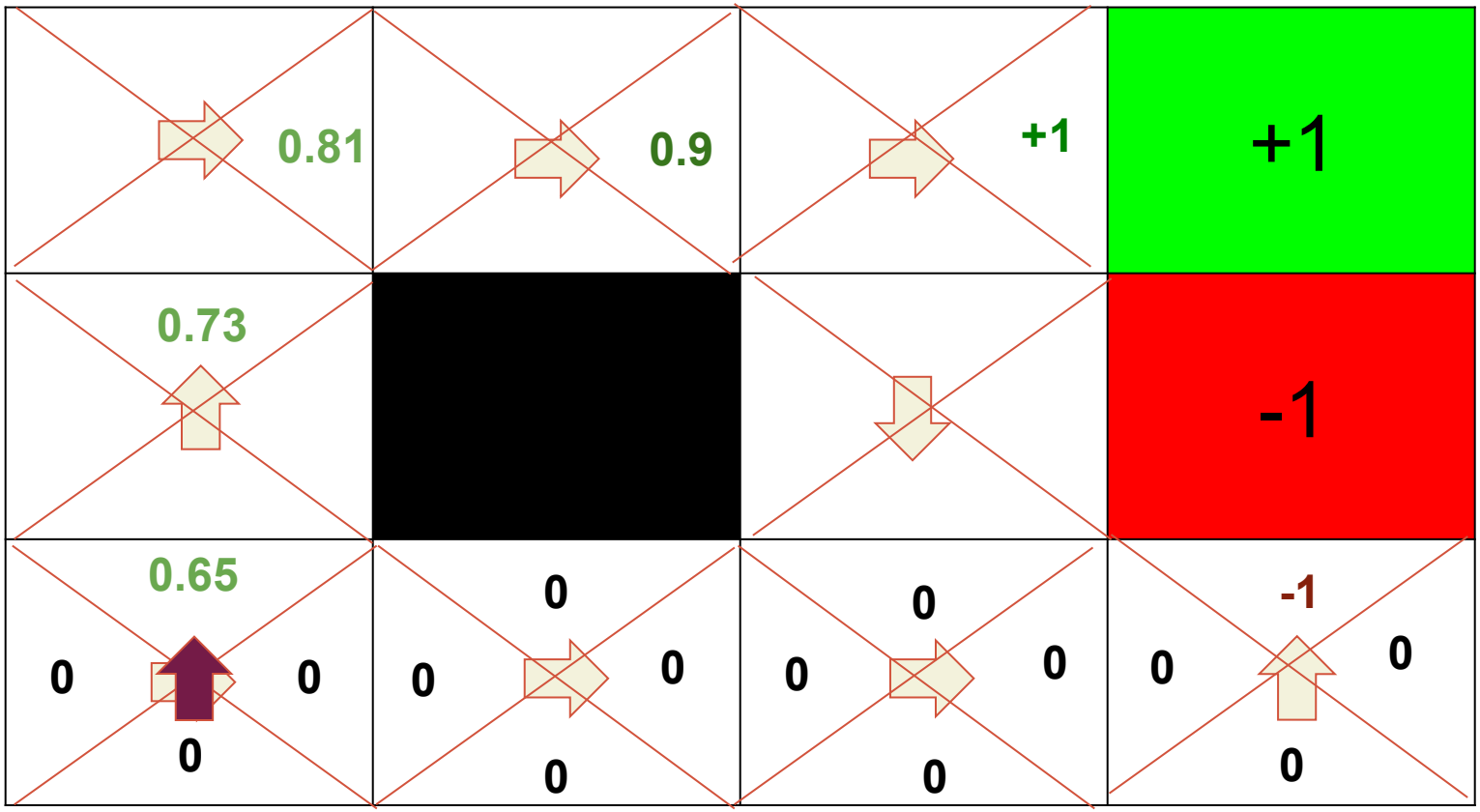


Bootstrap Estimator

$$Q(s,a) \approx r_0 + \gamma \max_b Q(s',b)$$



Policy Improvement



Bias and Variance in RL

What is bias of V_π estimation?

- Let $\hat{V}_\pi(s)$ be an estimate of $V_\pi(s)$
- $\hat{V}_\pi(s)$ is unbiased if:

$$\mathbb{E}_s\left[\hat{V}_\pi(s) - V_\pi(s)\right] = 0$$

What is variance of $\hat{V}_\pi(s)$ estimation?

$$\mathbb{V}\mathrm{ar}\left[\hat{V}_\pi(s)\right] = \mathbb{E}_s\left[(\hat{V}_\pi(s) - \mathbb{E}_s\left[\hat{V}_\pi(s)\right])^2\right]$$

- High if $\hat{V}_\pi(s)$ fluctuates a lot

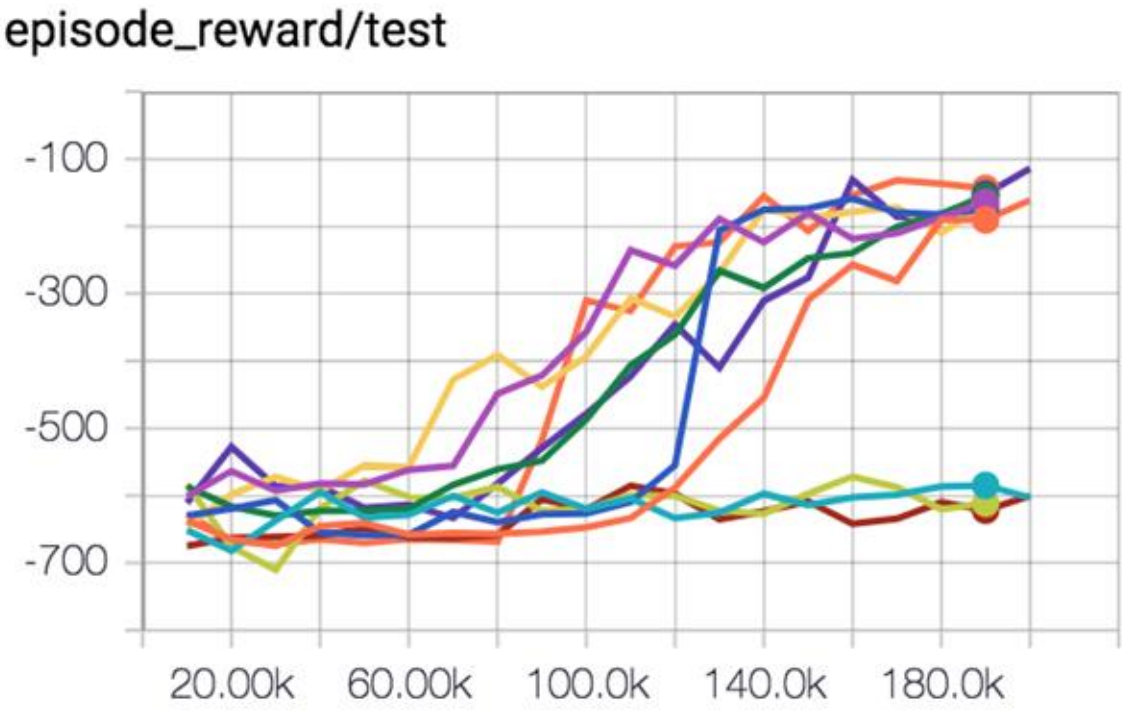
Bias and Variance

- Monte-Carlo estimate has high variance and low bias.
- Bootstrap estimate has higher bias but lower variance.

Problems with RL

Randomness

- Random initialization
- Random exploration
- Random environment

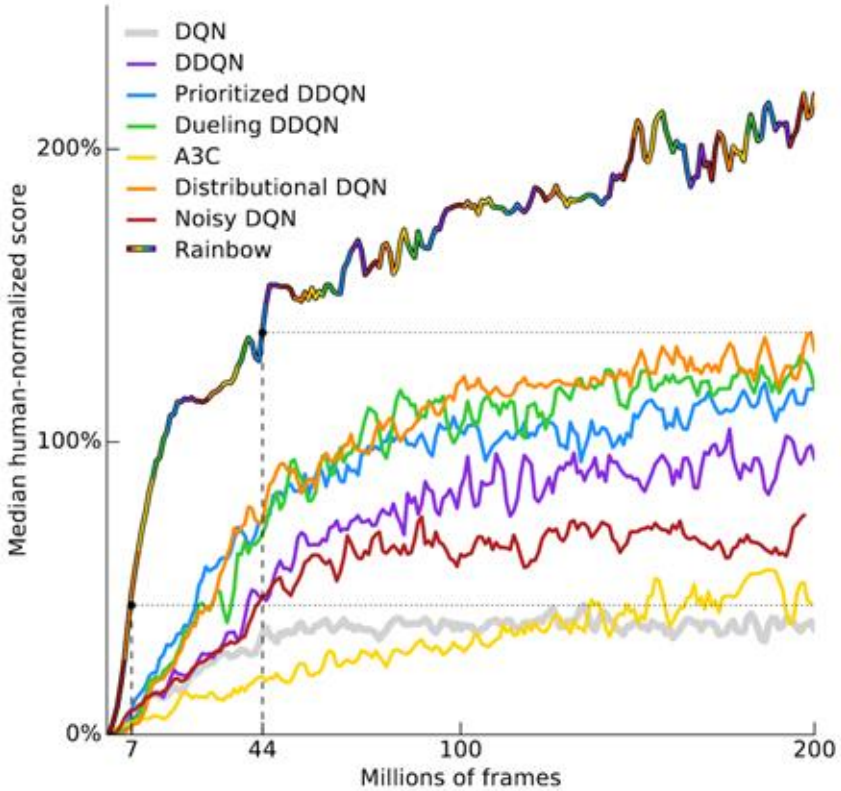


https://www.alexirpan.com/2018/02/14/rl-hard.html

Problems with (current) RL

Data inefficient

- Many use case can be better solved with supervised learning (efficiency and accuracy)



https://arxiv.org/abs/1710.02298

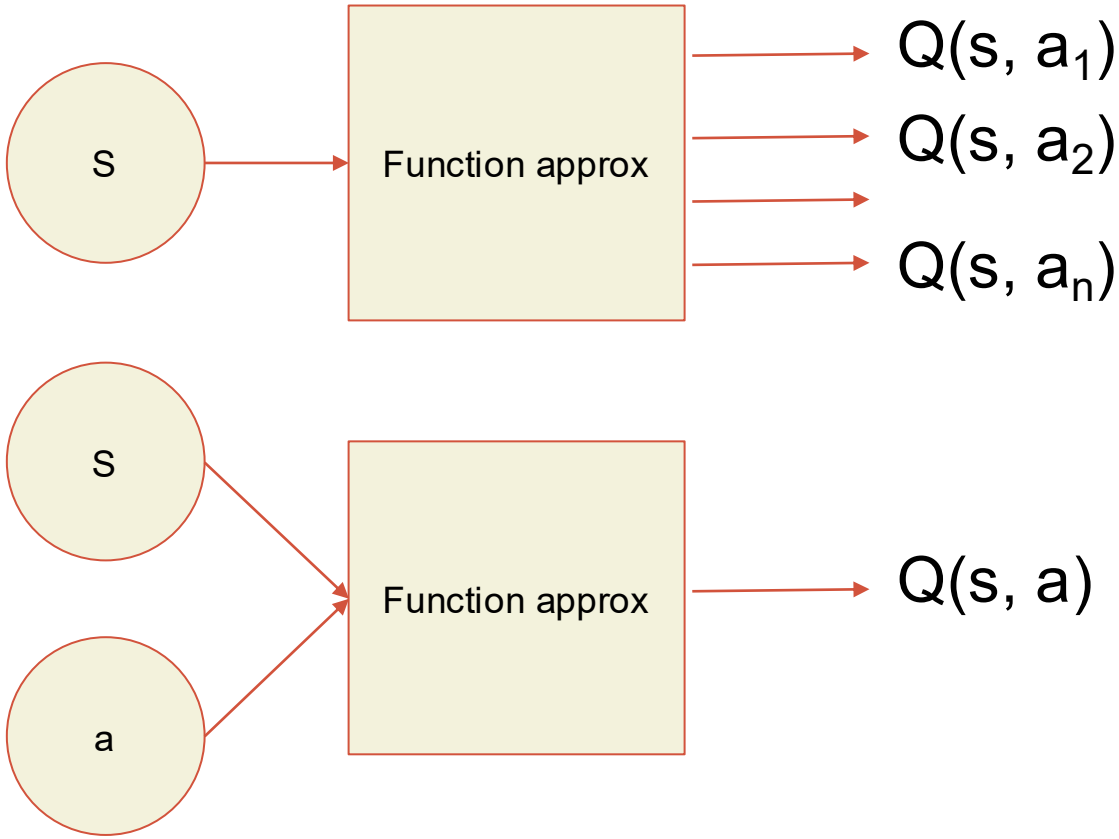
Function Approximator (FA)

- Tabular Q-value is impractical when the state-action space is large!
  - Need large memory
  - Impractical to fill up every cell

- Enter .. a function approximator

Function Approximator (FA)

- Instead of a table containing Q-value for every state and action, use a function that output Q-values.



Learning with FA

- With tabular Q-learning,
  - the act of learning = putting Q-value in the table

- With function approximator,
  - the act of learning = searching for the optimal parameters of the FA

Learning with FA

- How to adapt the parameters (weights) of the FA?
- Step 1: Define a loss function.
- Step 2: Optimise the weights to minimise the loss

Loss function

- What should be the loss function?
- Introducing Bellman's equations

$$V^{\pi}(s_t) = E_{\pi,P}[r_t + \gamma V^{\pi}(s_{t+1})]$$

$$Q^{\pi}(s_t) = E_{\pi,P}[r_t + \gamma Q^{\pi}(s_{t+1}, a_{t+1})]$$

- Bellman's optimality equations

$$Q^{*}(s_t) = E_{P}[r_t + \gamma \max_b Q^{*}(s_{t+1}, b)]$$

Loss function

- The Bellman's equation must hold for correct Q-value
- Rewrite the Bellman's optimality with our estimator (FA)

$$\hat{Q}(s_t) = E_P[r_t + \gamma \max_b \hat{Q}(s_{t+1}, b)]$$

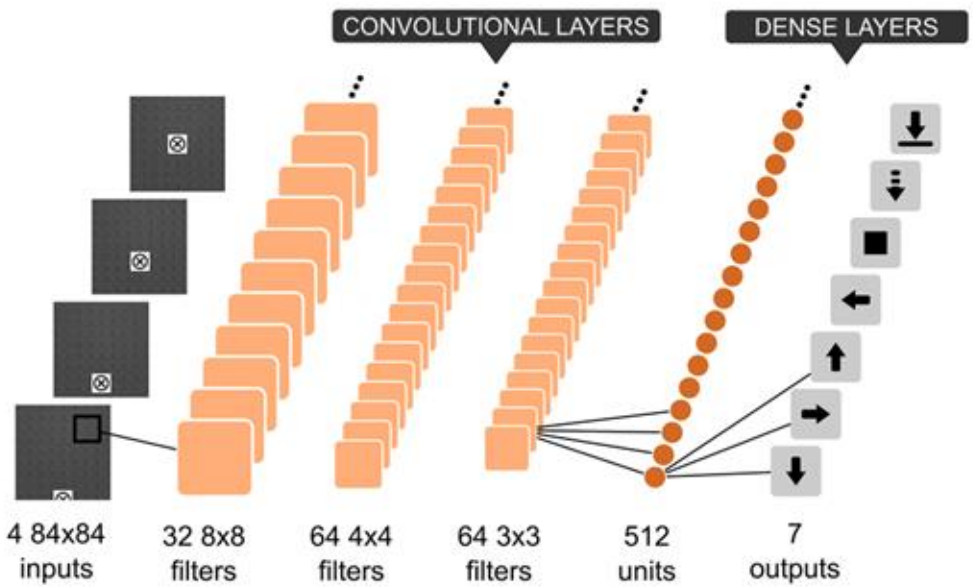
The estimator is correct if
the left hand side = right hand side

$$TD = r_t + \gamma \max_b \hat{Q}_\theta(s_{t+1}, b) - \hat{Q}_\theta(s_t)$$

``Temporal Difference error''

Temporal Difference Learning

- Use TD-error to guide learning
- Example
  - Deep Q-Networks (DQN)
  - Deep convolutional neural network as a function approximator
  - Optimize square TD-error



$$L(\theta) = (r_t + \gamma \max_b \hat{Q}_\theta(s_{t+1}, b) - \hat{Q}_\theta(s_t))^2$$

Policy-based methods

Policy gradient

Policy gradient

Q-Learning

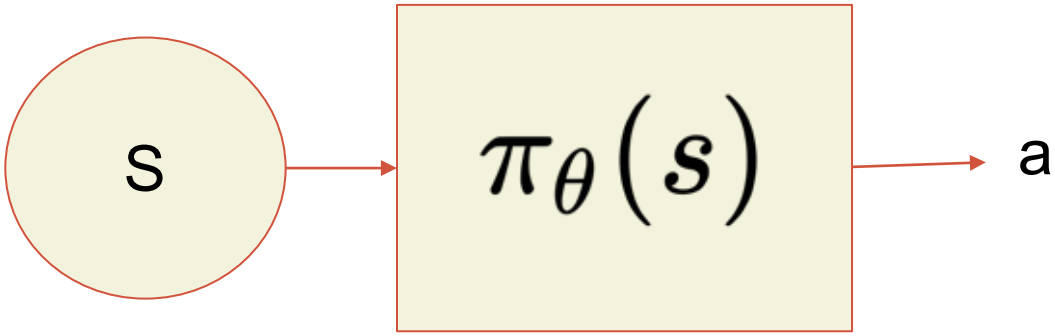
- policy is implicit
- if we already have Q, we have policy
- we just look at Q to get π

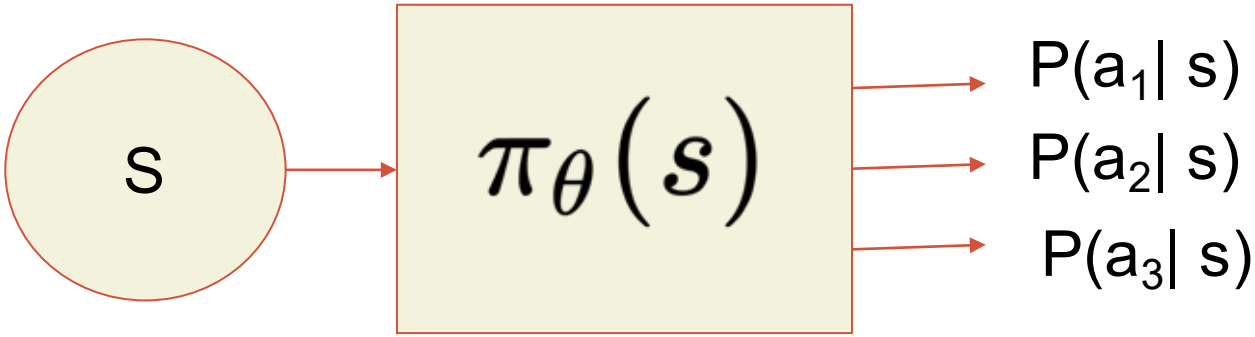
Policy gradient

- learns π directly explicitly
- use Q, V as a helper for learning π

Policy gradient

- Use Function Approximator to represent policy directly





Policy-based vs Value-based

Policy-based can learn continuous actions

Q-learning needs to $\text{argmax}_a Q(s,a)$

Policy-based can yield non-deterministic policy $P(a \mid s)$

Loss function for policy gradient

- Start-state objective

$$J(\theta) = E_{\pi(\theta)}[G|s_0] = V^{\pi}(s_0)$$

- Average-reward objective

$$J(\theta) = \sum_s d^{\pi}(s) \sum_a \pi(s,a) r(s,a)$$

* **d** is a stationary distribution of a Markov chain.

- One way to optimise these objectives is to use SGD.

Computing the gradient

Let's try to compute the gradient of the start-state objective

$$J(\theta) = E_{\pi_\theta}[G|s_0]$$

To evaluate this expectation, maybe we could try a one-sample Monte-Carlo estimator:

$$J(\theta) \approx r_0 + \gamma r_1 + \gamma^2 r_3 + \ldots$$

$$\nabla J(\theta) \approx \nabla_\theta [r_0 + \gamma r_1 + \gamma^2 r_3 + \ldots] \times$$

Doesn't quite work? The evaluated value does not depend on θ . Gradient can't be computed.

Computing the gradient

Maybe we can try change θ a little bit and find the difference?

$$J(\theta) \approx r_0 + \gamma r_1 + \gamma^2 r_3 + \dots$$

$$J(\theta + \delta\theta) \approx r'_0 + \gamma r'_1 + \gamma^2 r'_3 + \dots$$

$$\nabla J = \frac{J(\theta) - J(\theta + \delta)}{\delta}$$

Could work? But...

Looks very expensive and noisy to compute!

Maybe there is a better way?

Policy gradient

Let's start from the average-reward objective

$$J(\theta) = \sum_s d^\pi(s) \sum_a \pi_\theta(s,a) r(s,a)$$

For simplicity let's assume d(s) does not depend on θ

$$J(\theta) = \sum_s d(s) \sum_a \pi_\theta(s,a) r(s,a)$$

$$\nabla_\theta J(\theta) = \sum_s d(s) \sum_a \nabla_\theta \pi_\theta(s,a) r(s,a)$$

Almost there...

Policy gradient

REINFORCE trick!

$$\nabla_\theta J(\theta) = \sum_s d(s) \sum_a \nabla_\theta \pi_\theta(s,a) r(s,a)$$

$$\nabla_\theta J(\theta) = \sum_s d(s) \sum_a \pi_\theta \nabla_\theta \log \pi_\theta(s,a) r(s,a)$$

We get this by sampling a playthrough using current policy (on-policy)

$$\nabla_\theta J(\theta) = E_\pi[\nabla_\theta \log \pi_\theta(s,a) r(s,a)]$$

$$\nabla_\theta J(\theta) \approx \nabla_\theta \log \pi_\theta(s,a) r(s,a)$$

Policy gradient theorem

There is a theorem…called policy gradient theorem say that we can replace $r(s,a)$ with $Q(s,a)$

$$\nabla_\theta J(\theta) = E_{\pi_\theta}[Q^{\pi_\theta}(s,a)\nabla_\theta log\pi_\theta(s,a)]$$

What is the gradient doing?

Goal maximize rewards

Push **π** towards directions of higher Q(s,a)

$$\nabla_\theta J(\theta) = E_{\pi_\theta}[Q^{\pi_\theta}(s,a)\nabla_\theta log\pi_\theta(s,a)]$$

Q(1,1) = 5

Q(1,2) = 2

- **π**(1,2) will have a higher weight. Policy gets push towards action 2

Notes on Policy gradient

Also known as REINFORCE or likelihood ratio.

Used by other ML fields when original loss is not differentiable (-Q in this case). Push network to produce lower loss.

$$\nabla_\theta J(\theta) = E_{\pi_\theta}[Q^{\pi_\theta}(s,a)\nabla_\theta log\pi_\theta(s,a)]$$

Example of non-differentiable functions

argmax (not maxpool)

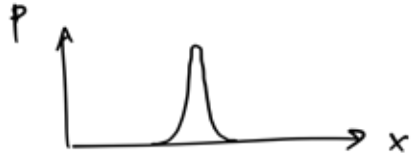
sampling

Encourage Exploration

- policy $\pi_\theta(a|s)$ could be too confident early
- Like, $\pi_\theta(a = a|s) = 1$
- This could lead to insufficient exploration
- Encourage exploration by "entropy term" $H(\pi_\theta)$
- We want to punish too low entropy
- New gradient rule: $\nabla_\theta J(\theta) \rightarrow \nabla_\theta J(\theta) + \nabla_\theta H(\pi_\theta)$

high entropy

low entropy



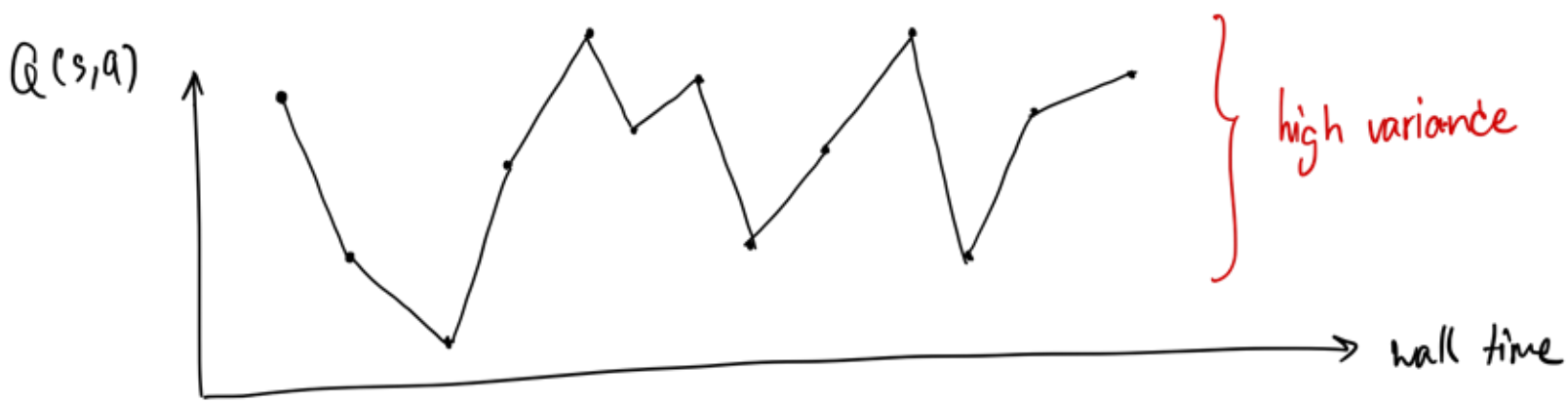
Baselines

$Q_\pi(s,a)$ has high variance

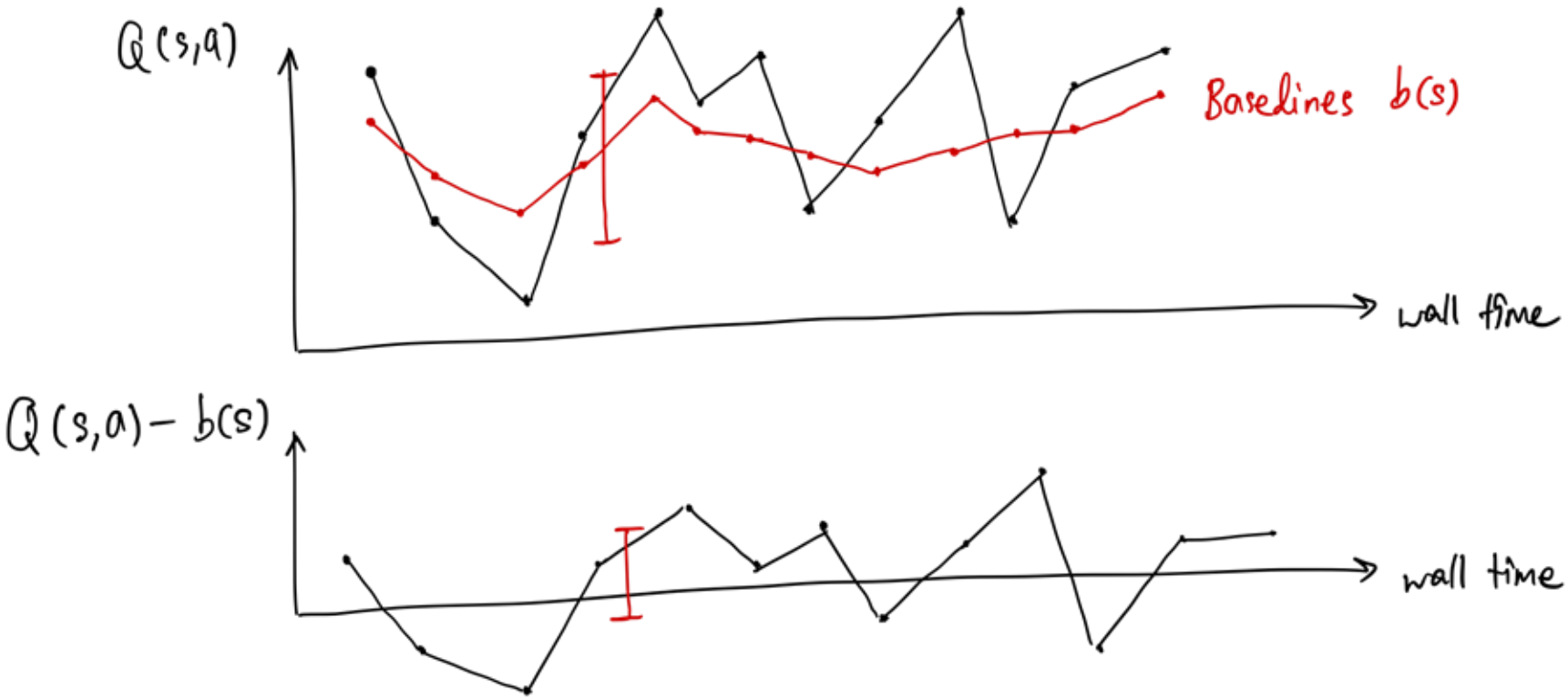
$Q_\pi(s,a)\nabla_\theta \log \pi_\theta(a|s) = \nabla_\theta J(\theta)$ is very noisy

Slow down training a lot!

We need to reduce variance to speed up the training



Baselines reduce variance



What is a good b(s)?

$V_\pi(s)$ is a convenient choice

Advantage Function

When we use V(s) as baseline:

$$A(s,a) = Q(s,a) - V(s)$$

We call this the **advantage function**.

It tells the relative value of the actions.

Lower variance than using absolute value of the actions.

$$\nabla J(\theta) = \mathbb{E}_{s,a}\left[A(s,a)\nabla \log \pi_\theta(a|s)\right]$$

Also called A2C

Noisy updates

- In RL, the updates can be noisy. Our learning is based on Q, V, and A and might not be an accurate estimate if the policy changes too much.
- Limit the update to a certain size? Use KL divergence that new policy shouldn't be that different from old policy

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log\left(\frac{P(x)}{Q(x)}\right)$$

$$\nabla J(\theta) = \mathbb{E}_{s,a}\left[A(s,a)\nabla \log \pi_\theta(a|s)\right]$$

$$J(\theta') - J(\theta) = \sum_t \mathbb{E}_{s_t \sim P'(s_t)}\left[\mathbb{E}_{a_t \sim \pi}\left[\frac{\pi'(a_t|s_t)}{\pi(a_t|s_t)}\gamma^t A^\pi(s_t, a_t)\right]\right]$$

We want to make KL(new || old) $\leqslant$ e

We can solve this using Lagrange multipliers and lots of maths. This is called **Trust Region Policy Optimization (TRPO)**

Proximal Policy Optimization (PPO)

- PPO is just a policy gradient update with Advantage function and clipping the value so that the model does not move outside of the trusted region

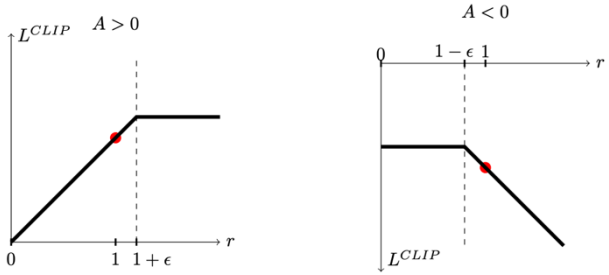
$$\min\left(\frac{\pi_\theta'(a|s)}{\pi_\theta(a|s)}A^{\pi_{\theta'}}(s,a), \text{clip}\left(\frac{\pi_\theta'(a|s)}{\pi_\theta(a|s)}, 1-\epsilon, 1+\epsilon\right)A^{\pi_{\theta'}}(s,a)\right)$$

or

$$\min\left(\frac{\pi_\theta'(a|s)}{\pi_{\theta'}(a|s)}A^{\pi_{\theta'}}(s,a),\ (1+\epsilon)A^{\pi_{\theta'}}(s,a)\right) \quad \text{if } A \geq 0$$

$$\min\left(\frac{\pi_\theta'(a|s)}{\pi_{\theta'}(a|s)}A^{\pi_\theta}(s,a),\ (1-\epsilon)A^{\pi_{\theta'}}(s,a)\right) \quad \text{if } A < 0$$

- The min is used to clip values so that the loss no longer changes beyond e. This is used instead of an explicit KL. PPO is one of the defaults techniques used by OpenAI



https://arxiv.org/pdf/1707.06347

Recap PPO and RL

- RL uses a reward function to learn to optimize our model.
  - Key ingredient: environment (game = input+output) and a way to give reward (scores from output)
- PPO is a policy gradient method with tricks.
  - Key: what moves give me higher reward than before, try to change the gradients so that the move is more likely

On policy and off policy algorithms

Q Learning: $Q^*(s_t, a_t) = r_{t+1} + \max_a Q^*(s_{t+1}, a)$

- you need a_t, s_t, r_{t+1}, s_{t+1} to satisfy the above equation
- you can get (a, s, r, s') from <u>any</u> *policy*
- Q learning is *off-policy*

Policy gradient: $\nabla_\theta J(\theta) = \mathbb{E}\left[Q_\pi(s,a)\nabla_\theta \log \pi_\theta(a|s)\right]$

- you need s, a and $Q_\pi(s,a)$
- Q_π needs to be from the current policy
- **s, a** needs to come from <u>current</u> policy
- policy gradient is *on-policy*

Notes on on vs off policy

Off-policy can learn from any policy

- Can use old experience
- Can use expert experience
- Sample efficient

On-policy can only learn from current policy

- Slow

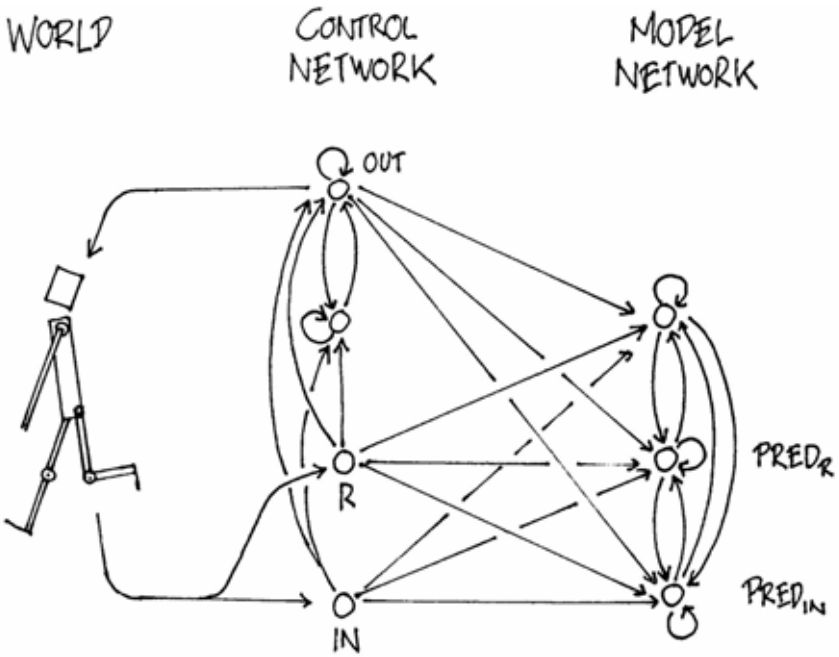
Model-based RL

RL's Model

RL's Model vs ML's Model

Think mental model that models the environment

Basically a model can guess the future or we have access to the environment so we can take alternative routes

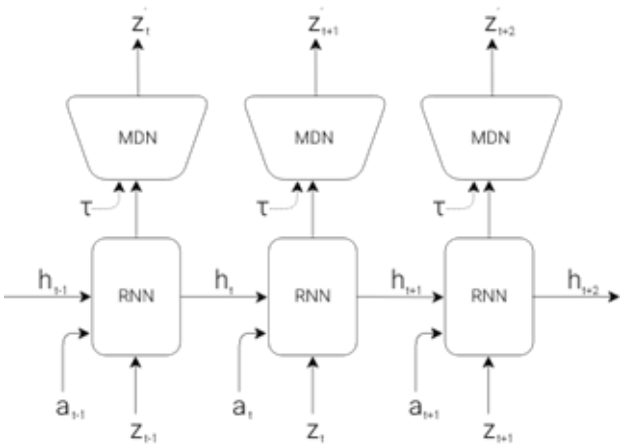


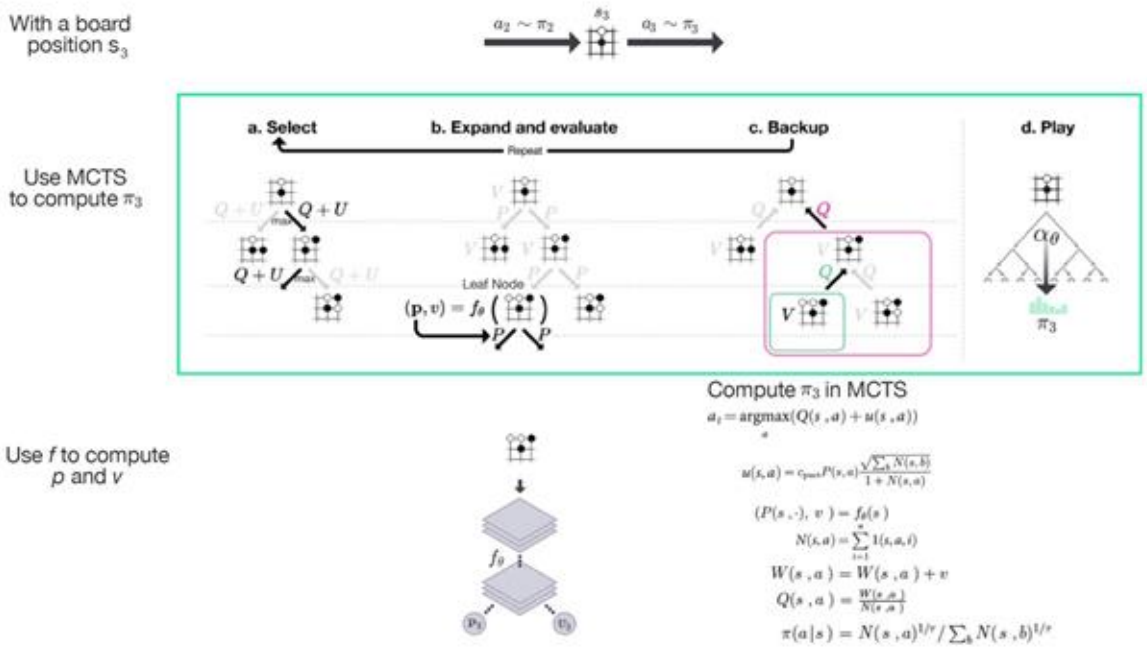
https://worldmodels.github.io/

RL's Model

Guess the future

Or access to the environment so we can take alternative routes





https://worldmodels.github.io/

https://medium.com/@jonathan_hui/monte-carlo-tree-search-mcts-in-alphago-zero-8a403588276a

Model-based RL

- In model-based RL, we first build the model of the environment
- Then use that model to directly search for the answer.
- The problem is … inaccurate model can give us bad policies…
- It is believed that if we can treat the uncertainty in the model correctly...model-based RL is the most efficient method!
- However, measuring uncertainty in the model is also very difficult.

Things to consider

When do we need RL?

- Your action affects the observation. Action has consequences (RL vs Bandit problem)
  - x_1 , a_1 $\Rightarrow$ x_2
- The target behavior is difficult to be directly hard-coded.
  - How to move a snake robot?
- Collection of the data of a target behavior is difficult.

AlphaGo and why it works so well

Properties of the game Go

Deterministic

Fully observable

Rules are known (Model completely known)

Static

Fits MDP (Markov property)

Credit assignment problem

- An action can have consequences further away in time
- Some movements might not have any effect on the outcome

10 time steps later

$r_t = 0$

$r_{t+10} = 1$

Sparse reward problem

- Another problem is when rewards are sparse.
- Since model-free RL is just learning the correlations of trajectories and rewards… when there is no reward, RL cannot learn.
- Can we make it better?
  - Curiosity + intrinsic motivation?
  - Curriculum learning?
  - Hierarchical RL?

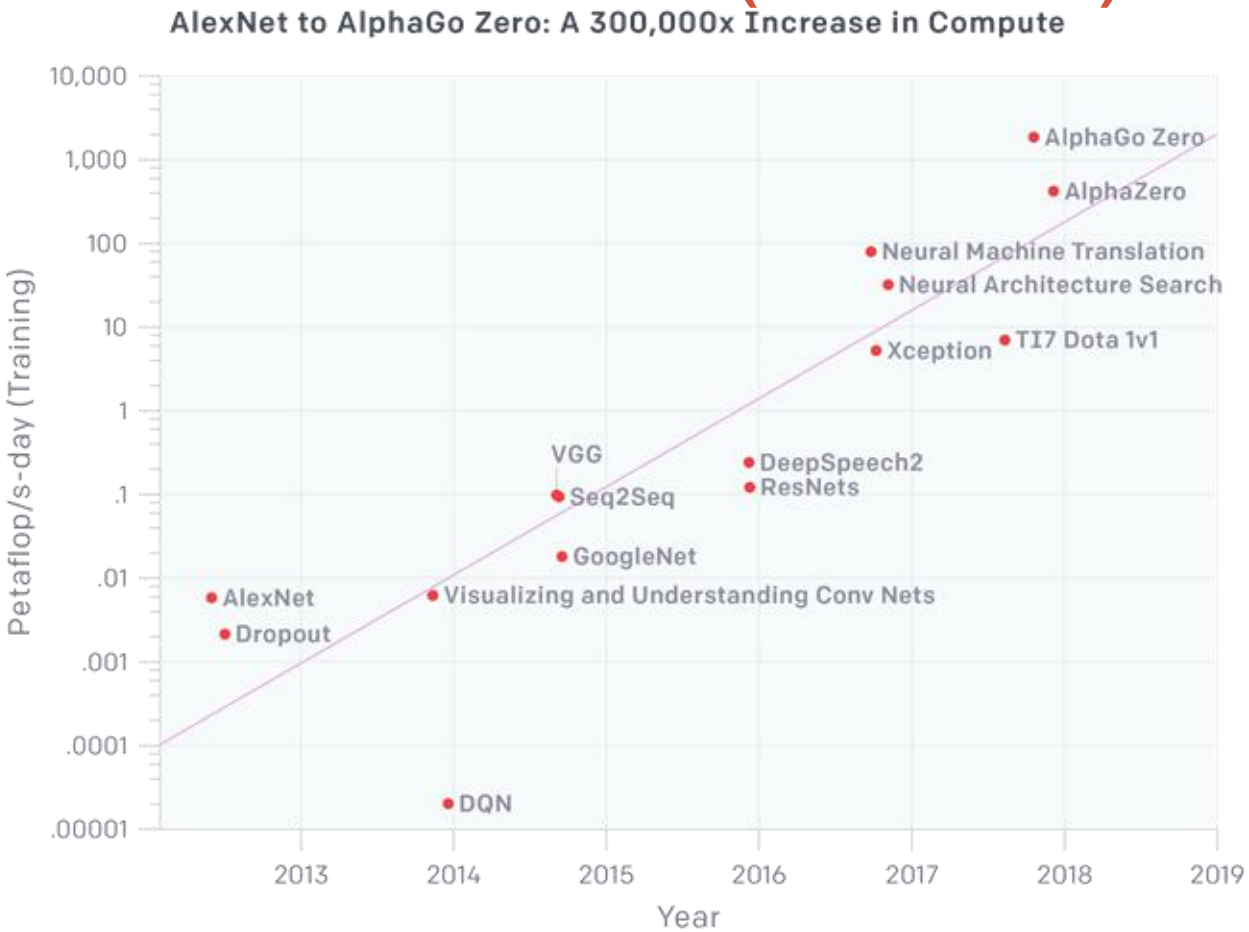
Designing the reward signal

- Reward design can be quite challenging..
- Naive reward design can lead to unexpected (cheating) behaviours!
- Example:

https://www.youtube.com/watch?v=tlOIHko8ySg

How chatGPT sounds so confident all the time is also a form of reward gaming.

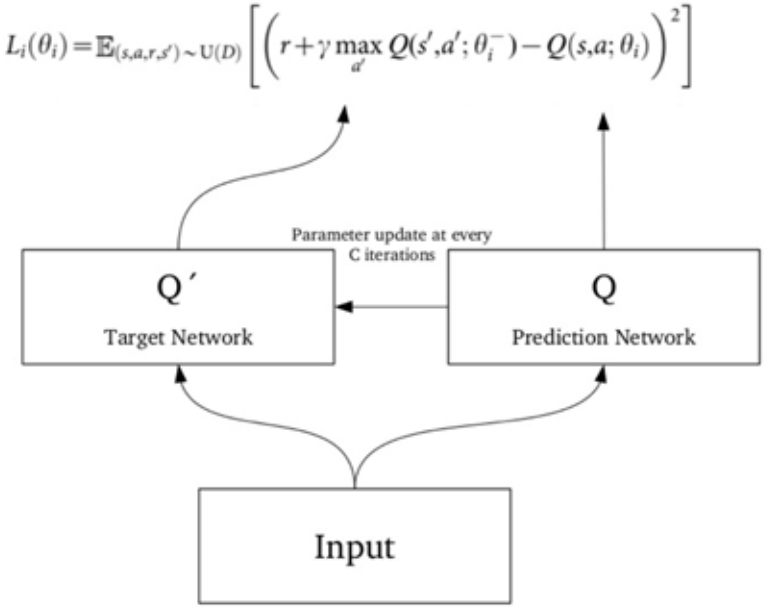
Problems with (current) RL



https://blog.openai.com/ai-and-compute/

Bias and Variance

- Bias and variance are really important in RL
- We want to reduce both of them as much as possible
  - DQN uses experience replay to reduce bias
  - DQN uses target network to reduce variance
  - Actor-Critic method uses baseline to reduce variance
  - Actor-Critic method uses parallel worker to reduce bias
  - etc.



https://www.slideshare.net/MuhammedKocaba/humanlevel-control-through-deep-reinforcement-learning-presentation

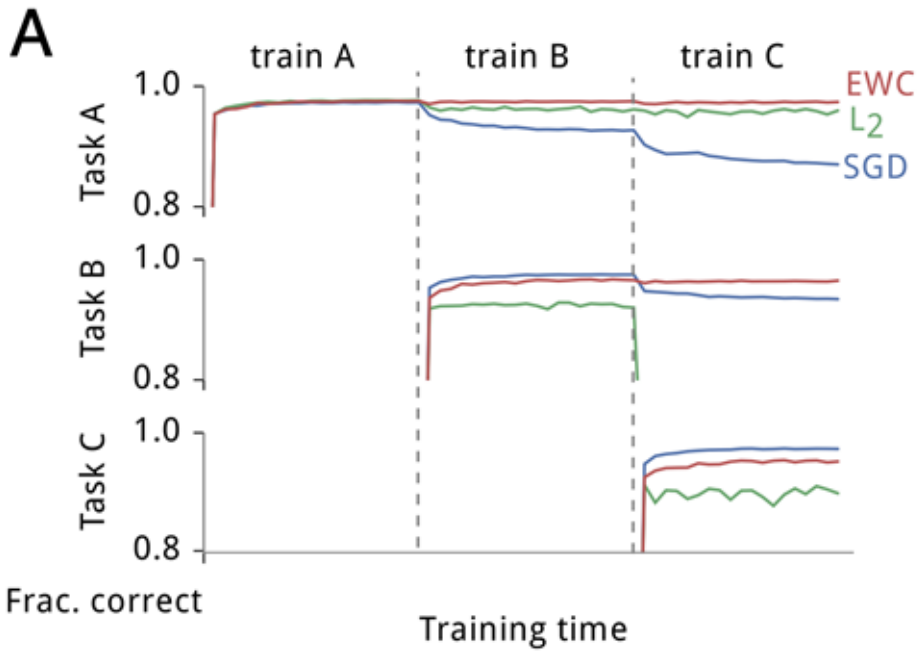
Exploration and Exploitation

- Many RL results assume that all states are visited infinitely often.
- Also, many RL algorithms are reduced into just an optimization problem.
- Therefore, nicely spread/informative data can help a lot!

- DQN uses epsilon-greedy for exploration
- Policy gradient uses entropy regularizer to encourage exploration

Optimisation problem

- Initialization problems
- Is SGD the best we can do?
- Catastrophic forgetting?



http://www.pnas.org/content/early/2017/03/13/1611835114.full.pdf?with-ds=yes

Reinforcement learning

Elements of RL

- Environment, Agent, State, and MDP

Estimating Q

- Monte-Carlo, Bootstrap
- Deep learning as a function approximator

Policy learning

- Q-learning
- TD learning
- Policy gradient, PPO

Concepts

- Exploration vs Exploitation

Further learning

- Chula RL course
  - https://www.youtube.com/playlist?list=PLcBOyD1N1T-PyNUNA77ITYNCAeAMGxV5I
- Deepmind RL course 2021
  - https://www.deepmind.com/learning-resources/reinforcement-learning-lecture-series-2021